



المحاضرة السابعة

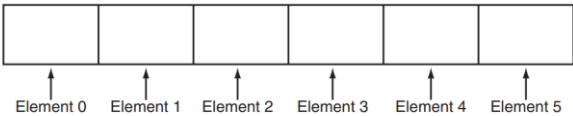
الأشعة والمصفوفات (البحث والفرز)

Arrays and Vectors (Searching and Sorting)

Topics for Lecture 7
7.1 Arrays Hold Multiple Values
7.2 Accessing Array Elements
7.3 The Range-Based for Loop
7.4 Processing Array Contents
7.5 Focus on Software Engineering: Using Parallel Arrays
7.6 Arrays as Function Arguments
7.7 Two-Dimensional Arrays
7.8 Arrays with Three or More Dimensions
7.9 Introduction to the STL vector
7.10 Focus on Software Engineering: Introduction to Search Algorithms
7.11 Focus on Software Engineering: Introduction to Sorting Algorithms
7.12 Sorting and Searching vectors

7.1 Arrays Hold Multiple Values

المفهوم: تسمح لك المصفوفة بتخزين قيم متعددة من نفس نوع البيانات والعمل عليها.

تم تصميم المتغيرات التي عملت بها حتى الآن لحمل قيمة واحدة فقط في كل مرة.	تعمل المصفوفة كمتغير يخزن مجموعة من القيم، جميعها من نفس النوع. تُخزن القيم معًا في مواقع ذاكرة متتالية. فيما يلي تعريف مصفوفة من الأعداد الصحيحة: <pre>int days[6];</pre>
Variables can hold one value at a time <pre>int count; Enough memory for 1 int 12314 float price; Enough memory for 1 float 56.981 char letter; Enough memory for 1 char A</pre>	An array with six elements days array: Enough memory for six int values 
	اسم هذه المصفوفة هو <code>days</code>. الرقم بين القوسين هو مُعلن حجم المصفوفة. ويشير إلى عدد العناصر، أو القيم، التي يمكن للمصفوفة استيعابها.
	يجب أن يكون مُعلن حجم المصفوفة تعبيرًا صحيحًا ثابتًا بقيمة أكبر من الصفر. يمكن أن يكون إما حرفيًا، كما في المثال السابق، أو ثابتًا مُسمًى، كما هو موضح في الشكل التالي: <pre>const int NUM_DAYS = 6; int days[NUM_DAYS];</pre>
	يمكن تعريف مصفوفات من أي نوع بيانات. فيما يلي جميع تعريفات المصفوفات الصحيحة: <pre>float temperatures[100]; // Array of 100 floats string names[10]; // Array of 10 string objects long units[50]; // Array of 50 long integers double sizes[1200]; // Array of 1200 doubles</pre>

Memory Requirements of Arrays

تعتمد كمية الذاكرة التي تستخدمها المصفوفة على نوع بياناتها وعدد عناصرها. مصفوفة `hours`، المُعرّفة هنا، هي مصفوفة مكونة من ستة عناصر `short`.

`short hours[6];`

في نظام نموذجي، تستهلك العناصر `short` بايتين من الذاكرة، لذا تشغل مصفوفة الساعات 12 بايتًا. كما هو موضح في الشكل 7-3.

Figure 7-3 Array memory usage

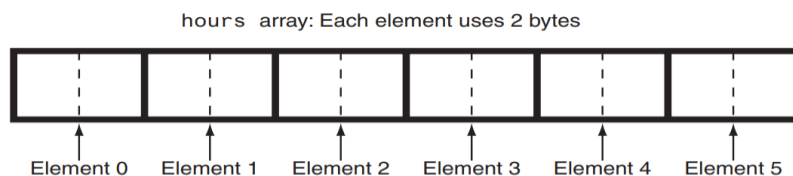


Table 7-1 Arrays and Their sizes

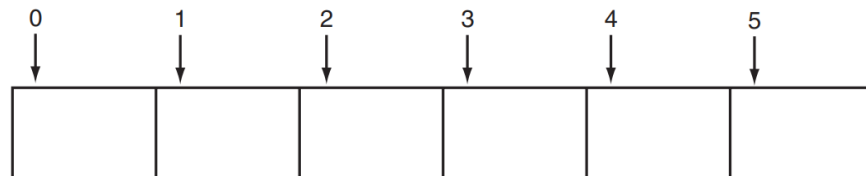
Array Definition	Number of Elements	Size of Each Element	Size of the Array
char letters[25];	25	1 byte	25 bytes
short rings[100];	100	2 bytes	200 bytes
int miles[84];	84	4 bytes	336 bytes
float temp[12];	12	4 bytes	48 bytes
double distance[1000];	1000	8 bytes	8000 bytes

7.2 Accessing Array Elements

المفهوم: يُخصَّص لكل عنصر من عناصر المصفوفة فهرس فريد (unique subscript). يُستخدم هذه الفهرس للوصول إلى العناصر.

Figure 7-4 Subscripts

Subscripts

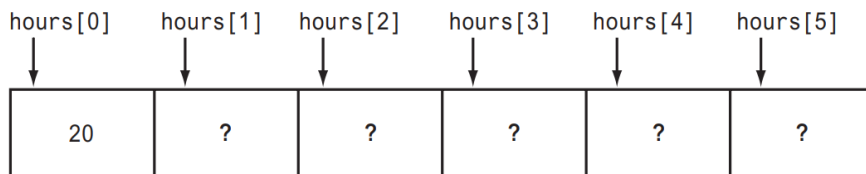


NOTE: Subscript numbering in C++ always starts at zero. The subscript of the last element in an array is one less than the total number of elements in the array. This means that in the array shown in Figure 7-4, the element `hours[6]` does not exist. `hours[5]` is the last element in the array.

hours[0] = 20;



NOTE: The expression `hours[0]` is pronounced “hours sub zero.” You would read this assignment statement as “hours sub zero is assigned twenty.”

Figure 7-5 Contents of the hours array

NOTE: Because values have not been assigned to the other elements of the array, question marks will be used to indicate that the contents of those elements are unknown. If an array is defined globally, all of its elements are initialized to zero by default. Local arrays, however, have no default initialization value.

The following statement stores the integer 30 in hours[3]:

```
hours[3] = 30;
```

Figure 7-6 shows the contents of the array after the previous statement executes.

Figure 7-6 Contents of the hours array

hours[0]	hours[1]	hours[2]	hours[3]	hours[4]	hours[5]
20	?	?	30	?	?



NOTE: Understand the difference between the array size declarator and a subscript. The number inside the brackets of an array definition is the size declarator. The number inside the brackets of an assignment statement or any statement that works with the contents of an array is a subscript.

Inputting and Outputting Array Contents

يمكن استخدام عناصر المصفوفة مع كائني cin و cout كأبي متغير آخر. يعرض البرنامج 1-7 المصفوفة hours المستخدمة لتخزين وعرض القيم التي يُدخلها المستخدم.

Program 7-1

```
1 // This program asks for the number of hours worked
2 // by six employees. It stores the values in an array.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int NUM_EMPLOYEES = 6;
9     int hours[NUM_EMPLOYEES];
10
11     // Get the hours worked by each employee.
12     cout << "Enter the hours worked by "
13          << NUM_EMPLOYEES << " employees: ";
14     cin >> hours[0];
15     cin >> hours[1];
16     cin >> hours[2];
17     cin >> hours[3];
18     cin >> hours[4];
19     cin >> hours[5];
20
21     // Display the values in the array.
22     cout << "The hours you entered are:";
23     cout << " " << hours[0];
24     cout << " " << hours[1];
25     cout << " " << hours[2];
26     cout << " " << hours[3];
27     cout << " " << hours[4];
28     cout << " " << hours[5] << endl;
29     return 0;
30 }
```

Program Output with Example Input Shown in Bold

Enter the hours worked by 6 employees: **20 12 40 30 30 15**
The hours you entered are: 20 12 40 30 30 15

يوضح الشكل 7-7 محتويات مصفوفة الساعات مع القيم التي أدخلها المستخدم في مثال الإخراج أعلاه.

Figure 7-7 Contents of the hours array

hours[0]	hours[1]	hours[2]	hours[3]	hours[4]	hours[5]
20	12	40	30	30	15

Stepping Through an Array

```
const int ARRAY_SIZE = 5;
int numbers[ARRAY_SIZE];
for (int count = 0; count < ARRAY_SIZE; count++)
    numbers[count] = 99;
```

Figure 7-8 Initialization, test, and update expressions

The variable count starts at 0, which is the first valid subscript value.

The loop ends when the variable count reaches 5, which is the first invalid subscript value.

```
for (count = 0; count < ARRAY_SIZE; count++)
    numbers[count] = 99;
```

The variable count is incremented after each iteration.

يمكن تبسيط البرنامج 7-1 باستخدام حلقتي for: واحدة لإدخال القيم في المصفوفة، والأخرى لعرض محتوياتها. يظهر ذلك في البرنامج 7-2.

Program 7-2

```
1 // This program asks for the number of hours worked
2 // by six employees. It stores the values in an array.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int NUM_EMPLOYEES = 6; // Number of employees
9     int hours[NUM_EMPLOYEES];    // Each employee's hours
10    int count;                    // Loop counter
11
12    // Input the hours worked.
13    for (count = 0; count < NUM_EMPLOYEES; count++)
14    {
15        cout << "Enter the hours worked by employee "
16              << (count + 1) << ": ";
17        cin >> hours[count];
18    }
19
20    // Display the contents of the array.
21    cout << "The hours you entered are:";
22    for (count = 0; count < NUM_EMPLOYEES; count++)
23        cout << " " << hours[count];
24    cout << endl;
25    return 0;
26 }
```

Program Output with Example Input Shown in Bold

```
Enter the hours worked by employee 1: 20 
Enter the hours worked by employee 2: 12 
Enter the hours worked by employee 3: 40 
Enter the hours worked by employee 4: 30 
Enter the hours worked by employee 5: 30 
Enter the hours worked by employee 6: 15 
The hours you entered are: 20 12 40 30 30 15
```



NOTE: You can use any integer expression as an array subscript. For example, the first loop in Program 7-2 could have been written like this:

```
for (count = 1; count <= NUM_EMPLOYEES; count++)
{
    cout << "Enter the hours worked by employee "
        << count << ": ";
    cin >> hours[count - 1];
}
```

In this code, the `cin` statement uses the expression `count - 1` as a subscript.

Array Initialization

كما هو الحال مع المتغيرات العادية، تتيح لك لغة C++ تهيئة عناصر المصفوفة عند إنشائها. إليك مثال:

```
const int MONTHS = 12;
```

```
int days[MONTHS] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

تُسمى سلسلة القيم الموجودة داخل الأقواس والمفصولة بفواصل بقائمة تهيئة (*initialization*) *(list)*.

Figure 7-9 The array after initialization

Subscripts											
0	1	2	3	4	5	6	7	8	9	10	11
31	28	31	30	31	30	31	31	30	31	30	31

Program 7-3

```
1 // This program displays the number of days in each month.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int MONTHS = 12;
8     int days[MONTHS] = { 31, 28, 31, 30,
9                          31, 30, 31, 31,
10                         30, 31, 30, 31 };
11
12     for (int count = 0; count < MONTHS; count++)
13     {
14         cout << "Month " << (count + 1) << " has ";
15         cout << days[count] << " days.\n";
16     }
17     return 0;
18 }
```

Program Output

```
Month 1 has 31 days.
Month 2 has 28 days.
Month 3 has 31 days.
Month 4 has 30 days.
Month 5 has 31 days.
Month 6 has 30 days.
Month 7 has 31 days.
Month 8 has 31 days.
Month 9 has 30 days.
Month 10 has 31 days.
Month 11 has 30 days.
Month 12 has 31 days.
```



NOTE: C++ allows you to spread the initialization list across multiple lines. Both of the following array definitions are equivalent:

```
double coins[5] = {0.05, 0.1, 0.25, 0.5, 1.0};
double coins[5] = {0.05,
                   0.1,
                   0.25,
                   0.5,
                   1.0};
```

يُظهر البرنامج 4-7 مثالاً بمصفوفة سلسلة يتم تهيئتها بالسلاسل.

Program 7-4

```
1 // This program initializes a string array.
2 #include<iostream>
3 #include<string>
4 using namespace std;
5
6 int main()
7 {
8     const int SIZE = 9;
9     string planets[SIZE] = { "Mercury", "Venus", "Earth", "Mars",
10                             "Jupiter", "Saturn", "Uranus",
11                             "Neptune", "Pluto (a dwarf planet)" };
12
13     cout << "Here are the planets:\n";
14
15     for (int count = 0; count < SIZE; count++)
16         cout << planets[count] << endl;
17     return 0;
18 }
```

Program Output

```
Here are the planets:
Mercury
Venus
Earth
Mars
Jupiter
Saturn
Uranus
Neptune
Pluto (a dwarf planet)
```

يُظهر البرنامج 5-7 مصفوفة أحرف يتم تهيئتها بالأحرف العشرة الأولى من الأبجدية. تُستخدم المصفوفة بعد ذلك لعرض رموز ASCII الخاصة بتلك الأحرف.

Program 7-5

```
1 // This program uses an array of ten characters to store the
2 // first ten letters of the alphabet. The ASCII codes of the
3 // characters are displayed.
4 #include <iostream>
5 using namespace std;
6
7 int main()
8 {
9     const int NUM_LETTERS = 10;
10    char letters[NUM_LETTERS] = {'A', 'B', 'C', 'D', 'E',
11                                'F', 'G', 'H', 'I', 'J'};
12
```

```

13     cout << "Character" << "\t" << "ASCII Code\n";
14     cout << "-----" << "\t" << "-----\n";
15     for (int count = 0; count < NUM_LETTERS; count++)
16     {
17         cout << letters[count] << "\t\t";
18         cout << static_cast<int>(letters[count]) << endl;
19     }
20     return 0;
21 }

```

Program Output

Character	ASCII Code
-----	-----
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74

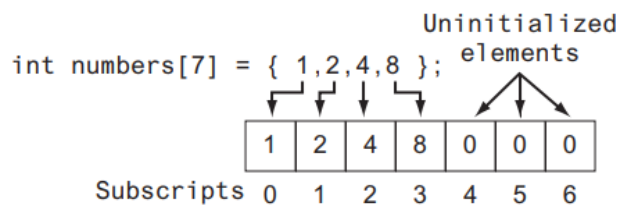


NOTE: An array's initialization list cannot have more values than the array has elements.

Partial Array Initialization

عند تهيئة مصفوفة، لا تطلب C++ قيمة لكل عنصر. من الممكن تهيئة جزء فقط من المصفوفة، مثل:

Partial array initialization



سيتم ضبط العناصر غير المهيأة على الصفر. ستحتوي العناصر غير المهيأة لمصفوفة نصية على سلاسل نصية فارغة.

إذا تركت عنصراً دون تهيئة، فيجب عليك أيضاً ترك جميع العناصر التي تليه دون تهيئة. لا نتيح لغة C++ إمكانية تخطي العناصر في قائمة التهيئة. على سبيل المثال، ما يلي غير قانوني:

```
int numbers[6] = {2, 4, , 8, , 12}; // NOT Legal!
```

يُظهر البرنامج 6-7 محتويات مصفوفة الأرقام بعد تهيئتها جزئيًا.

Program 7-6

```
1 // This program has a partially initialized array.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int SIZE = 7;
8     int numbers[SIZE] = {1, 2, 4, 8}; // Initialize first 4 elements
9
10    cout << "Here are the contents of the array:\n";
11    for (int index = 0; index < SIZE; index++)
12        cout << numbers[index] << " ";
13
14    cout << endl;
15    return 0;
16 }
```

Program Output

Here are the contents of the array:
1 2 4 8 0 0 0

Implicit Array Sizing

من الممكن تعريف مصفوفة دون تحديد حجمها، شريطة توفير قائمة تهيئة. تُنشئ لغة C++ المصفوفة تلقائيًا بحجم كافٍ لاستيعاب جميع قيم التهيئة. على سبيل المثال، يُنشئ التعريف التالي مصفوفة بخمسة عناصر:

```
double ratings[] = {1.0, 1.5, 2.0, 2.5, 3.0};
```



NOTE: You *must* provide an initialization list if you leave out an array's size declarator. Otherwise, the compiler doesn't know how large to make the array.

Reading Data from a File into an Array

قراءة محتويات ملف في مصفوفة أمرٌ بسيط: افتح الملف واستخدم حلقةً برمجيةً لقراءة كل عنصر منه، مع تخزين كل عنصر في عنصر مصفوفة. يجب أن تتكرر الحلقة حتى تمتلئ المصفوفة أو يصل الملف إلى نهايته. يوضح البرنامج 7-7 ذلك بفتح ملف يحتوي على 10 أرقام مخزنة، ثم قراءة محتويات الملف في مصفوفة.

Program 7-7

```
1 // This program reads data from a file into an array.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     const int ARRAY_SIZE = 10; // Array size
9     int numbers[ARRAY_SIZE]; // Array with 10 elements
10    int count = 0; // Loop counter variable
11    ifstream inputFile; // Input file stream object
12
13    // Open the file.
14    inputFile.open("TenNumbers.txt");
15
16    // Read the numbers from the file into the array.
17    while (count < ARRAY_SIZE && inputFile >> numbers[count])
18        count++;
19
20    // Close the file.
21    inputFile.close();
22
23    // Display the numbers read:
24    cout << "The numbers are: ";
25    for (count = 0; count < ARRAY_SIZE; count++)
26        cout << numbers[count] << " ";
27    cout << endl;
28    return 0;
29 }
```

Program Output

The numbers are: 101 102 103 104 105 106 107 108 109 110

Writing the Contents of an Array to a File

كتابة محتويات مصفوفة إلى ملف أمر سهل أيضاً. استخدم حلقة برمجية للانتقال بين عناصر المصفوفة، وكتابة محتوياتها إلى ملف. يوضح البرنامج 7-8 ذلك.

Program 7-8

```
1 // This program writes the contents of an array to a file.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     const int ARRAY_SIZE = 10; // Array size
9     int numbers[ARRAY_SIZE]; // Array with 10 elements
10    int count; // Loop counter variable
11    ofstream outputFile; // Output file stream object
12
13    // Store values in the array.
14    for (count = 0; count < ARRAY_SIZE; count++)
15        numbers[count] = count;
16
17    // Open a file for output.
18    outputFile.open("SavedNumbers.txt");
19
20    // Write the array contents to the file.
21    for (count = 0; count < ARRAY_SIZE; count++)
22        outputFile << numbers[count] << endl;
23
24    // Close the file.
25    outputFile.close();
26
27    // That's it!
28    cout << "The numbers were saved to the file.\n ";
29    return 0;
30 }
```

Program Output

The numbers were saved to the file.

Contents of the File

SavedNumbers.txt

0
1
2
3
4
5
6
7
8
9

7.3 The Range-Based for Loop

المفهوم: حلقة **for** القائمة على النطاق (**range-based for loop**) هي حلقة تتكرر مرة واحدة لكل عنصر في المصفوفة. في كل مرة تتكرر فيها الحلقة، تنسخ عنصرًا من المصفوفة إلى متغير. تم تقديم حلقة **for** القائمة على النطاق في **C++ 11**.

صُممت حلقة **for** القائمة على النطاق للعمل مع متغير مُدمج يُعرف باسم متغير النطاق. في كل مرة تتكرر فيها حلقة **for** القائمة على النطاق، تنسخ عنصر مصفوفة إلى متغير النطاق. على سبيل المثال، في المرة الأولى التي تتكرر فيها الحلقة، سيحتوي متغير النطاق على قيمة العنصر 0، وفي المرة الثانية التي تتكرر فيها الحلقة، سيحتوي متغير النطاق على قيمة العنصر 1، وهكذا.

هذا هو التنسيق العام للحلقة المستندة إلى النطاق:

```
for (dataType rangeVariable : array)
    statement;
```

لنلق نظرةً عن كثب على النحو التالي:

- **dataType** هو نوع بيانات متغير النطاق. يجب أن يكون مطابقًا لنوع بيانات عناصر المصفوفة، أو نوعًا يمكن تحويل العناصر إليه تلقائيًا.
- **rangeVariable** هو اسم متغير النطاق. سيستقبل هذا المتغير قيمة عنصر مصفوفة مختلف خلال كل تكرار للحلقة. خلال التكرار الأول للحلقة، سيستقبل قيمة العنصر الأول؛ وخلال التكرار الثاني، سيستقبل قيمة العنصر الثاني، وهكذا.
- **array** هو اسم المصفوفة التي تريد أن تعمل عليها الحلقة. ستتكرر الحلقة مرة واحدة لكل عنصر في المصفوفة.
- **statement** هي عبارة تُنفذ أثناء تكرار الحلقة. إذا كنت بحاجة إلى تنفيذ أكثر من عبارة واحدة في الحلقة، فضع العبارات بين أقواس معقوفة.

على سبيل المثال، افترض أن لديك تعريف المصفوفة التالي:

```
int numbers[] = { 3, 6, 9 };
```

يمكنك استخدام حلقة **for** المستندة إلى النطاق التالية لعرض محتويات مجموعة الأرقام:

<pre>for (int val : numbers) cout << val << endl;</pre>	The code's output will be as follows: <pre>3 6 9</pre>
---	---

فيما يلي مثال على حلقة for تعتمد على النطاق وتنفذ أكثر من عبارة واحدة في نص الحلقة:	
<pre>int[] numbers = { 3, 6, 9 }; for (int val : numbers) { cout << "The next value is "; cout << val << endl; }</pre>	This code will produce the following output: <pre>The next value is 3 The next value is 6 The next value is 9</pre>

يمكنك استخدام الكلمة المفتاحية auto لتحديد نوع بيانات متغير النطاق. إليك مثال:	
<pre>int numbers [] = { 3, 6, 9 }; for (auto val : numbers) cout << val << endl;</pre>	The code's output will be as follows: <pre>3 6 9</pre>

يوضح البرنامج 10-7 حلقة for القائمة على النطاق من خلال عرض عناصر مصفوفة int:

Program 7-10

```
1 // This program demonstrates the range-based for loop.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Define an array of integers.
8     int numbers[] = { 10, 20, 30, 40, 50 };
9
10    // Display the values in the array.
11    for (int val : numbers)
12        cout << val << endl;
13
14    return 0;
15 }
```

(program output continues)

Program 7-10 (continued)

Program Output

```
10
20
30
40
50
```

يُظهر البرنامج 11-7 مثالاً آخر على حلقة for القائمة على النطاق. يعرض هذا البرنامج عناصر مصفوفة نصية.

Program 7-11

```
1 // This program demonstrates the range-based for loop.
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 int main()
7 {
8     string planets[] = { "Mercury", "Venus", "Earth", "Mars",
9                           "Jupiter", "Saturn", "Uranus",
10                          "Neptune", "Pluto (a dwarf planet)" };
11
12     cout << "Here are the planets:\n";
13
14     // Display the values in the array.
15     for (string val : planets)
16         cout << val << endl;
17
18     return 0;
19 }
```

Program Output

```
Here are the planets:
Mercury
Venus
Earth
Mars
Jupiter
Saturn
Uranus
Neptune
Pluto (a dwarf planet)
```

Modifying an Array with a Range-Based for Loop

أثناء تنفيذ حلقة for القائمة على النطاق، يحتوي متغير النطاق الخاص بها على نسخة من عنصر مصفوفة فقط. نتيجة لذلك، لا يمكنك استخدام حلقة for القائمة على النطاق لتعديل محتويات مصفوفة إلا إذا أعلنت متغير النطاق كمرجع (reference). لإعلان متغير النطاق كمتغير مرجعي، ما عليك سوى كتابة علامة (&) قبل اسمه في رأس الحلقة. يوضح البرنامج 12-7 مثالاً.

Program 7-12

```
1 // This program uses a range-based for loop to
2 // modify the contents of an array.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int SIZE = 5;
9     int numbers[5];
10
11     // Get values for the array.
12     for (int &val : numbers)
13     {
14         cout << "Enter an integer value: ";
15         cin >> val;
16     }
17
18     // Display the values in the array.
19     cout << "Here are the values you entered:\n";
20     for (int val : numbers)
21         cout << val << endl;
22
23     return 0;
24 }
```

Program Output with Example Input Shown in Bold

```
Enter an integer value: 1 
Enter an integer value: 2 
Enter an integer value: 3 
Enter an integer value: 4 
Enter an integer value: 5 

Here are the values you entered:
1
2
3
4
5
```

The Range-Based for Loop versus the Regular for Loop

يمكن استخدام حلقة for القائمة على النطاق في أي حالة تحتاج فيها إلى التنقل بين عناصر المصفوفة، دون الحاجة إلى استخدام فهرس العناصر. مع ذلك، لن تعمل هذه الحلقة في الحالات التي تحتاج فيها إلى فهرس العناصر لغرض ما. في هذه الحالات، ستحتاج إلى استخدام حلقة for الاعتيادية.



NOTE: You can use the auto key word with a reference range variable. For example, the code in lines 12 through 16 in Program 7-12 could have been written like this:

```
for (auto &val : numbers)
{
    cout << "Enter an integer value: ";
    cin >> val;
}
```

7.4 Processing Array Contents

المفهوم: تتم معالجة عناصر المصفوفة الفردية مثل أي نوع آخر من المتغيرات.

لا تختلف معالجة عناصر المصفوفة عن معالجة المتغيرات الأخرى. على سبيل المثال، العبارة التالية تضرب hours[3] في معدل المتغير rate:

```
pay = hours[3] * rate;
```

وفيما يلي أمثلة على عمليات ما قبل الزيادة وما بعدها على عناصر المصفوفة:

```
int score[5] = {7, 8, 9, 10, 11};
```

```
++score[2]; // Pre-increment operation on the value in score[2]
```

```
score[4]++; // Post-increment operation on the value in score[4]
```



NOTE: When using increment and decrement operators, be careful not to confuse the subscript with the array element. For example, the following statement decrements the variable count, but does nothing to the value in amount[count].

```
amount[count--];
```

To decrement the value stored in amount[count], use the following statement:

```
amount[count]--;
```

يوضح البرنامج 7-13 استخدام عناصر المصفوفة في عبارة رياضية بسيطة. تمر حلقة برمجية عبر كل عنصر من عناصر المصفوفة، مستخدمةً هذه العناصر لحساب إجمالي رواتب خمسة موظفين.

Program 7-13

```
1 // This program stores, in an array, the hours worked by
2 // employees who all make the same hourly wage.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     const int NUM_EMPLOYEES = 5; // Number of employees
10    int hours[NUM_EMPLOYEES];     // Array to hold hours
11    double payrate;                // Hourly pay rate
12    double grossPay;              // To hold the gross pay
13
14    // Input the hours worked.
15    cout << "Enter the hours worked by ";
16    cout << NUM_EMPLOYEES << " employees who all\n";
17    cout << "earn the same hourly rate.\n";
18    for (int index = 0; index < NUM_EMPLOYEES; index++)
19    {
20        cout << "Employee #" << (index + 1) << ": ";
21        cin >> hours[index];
22    }
23
```

```

24 // Input the hourly rate for all employees.
25 cout << "Enter the hourly pay rate for all the employees: ";
26 cin >> payrate;
27
28 // Display each employee's gross pay.
29 cout << "Here is the gross pay for each employee:\n";
30 cout << fixed << showpoint << setprecision(2);
31 for (int index = 0; index < NUM_EMPLOYEES; index++)
32 {
33     grossPay = hours[index] * payrate;
34     cout << "Employee #" << (index + 1);
35     cout << ": $" << grossPay << endl;
36 }
37 return 0;
38 }

```

Program Output with Example Input Shown in Bold

Enter the hours worked by 5 employees who all
earn the same hourly rate.

Employee #1: **5**
Employee #2: **10**

(program output continues)

Program 7-13 (continued)

```

Employee #3: 15 
Employee #4: 20 
Employee #5: 40 
Enter the hourly pay rate for all the employees: 12.75 
Here is the gross pay for each employee:
Employee #1: $63.75
Employee #2: $127.50
Employee #3: $191.25
Employee #4: $255.00
Employee #5: $510.00

```

ملاحظة: يُعرّف الكود التالي مصفوفتين صحيحتين: القيم الجديدة والقيم القديمة. القيم الجديدة غير مُهيأة، بينما القيم القديمة مُهيأة بالقيم 10، 100، 200، و300.

```

const int SIZE = 4;
int oldValues[SIZE] = {10, 100, 200, 300};
int newValues[SIZE];

```

الطريقة الوحيدة لإسناد مصفوفة بأخرى هي إسناد عناصر المصفوفتين كل على حدة. عادةً، يُفضل استخدام حلقة تكرارية، مثل:

```

for (int count = 0; count < SIZE; count++)
    newValues[count] = oldValues[count];

```

Printing the Contents of an Array

افترض أن لدينا تعريف المصفوفة التالي:

```

const int SIZE = 5;
int numbers [SIZE] = {10, 20, 30, 40, 50};

```

يجب عليك استخدام حلقة لعرض محتويات كل عنصر من عناصر المصفوفة، على النحو التالي:

```
for (int count = 0; count < SIZE; count++)  
    cout << numbers[count] << endl;
```

في C++ 11 والإصدارات الأحدث، يمكنك استخدام حلقة for القائمة على النطاق لعرض محتويات المصفوفة، كما هو موضح هنا:

```
for (int val : numbers)  
    cout << val << endl;
```

Summing the Values in a Numeric Array

لجمع قيم مصفوفة، يجب استخدام حلقة تكرارية مع متغير تراكمي. تضيف الحلقة قيمة كل عنصر من عناصر المصفوفة إلى المتغير. على سبيل المثال، افترض أن العبارات التالية تظهر في برنامج، وأن القيم مُخزنة في مصفوفة units:

```
const int NUM_UNITS = 24;  
int units[NUM_UNITS];
```

يستخدم الكود التالي حلقة for عادية لإضافة قيم كل عنصر في المصفوفة إلى متغير total. عند اكتمال الكود، سيحتوي total على مجموع وحدات عناصر المصفوفة.

```
int total = 0; // Initialize accumulator  
for (int count = 0; count < NUM_UNITS; count++)  
    total += units[count];
```

في إصدار C++ 11 والإصدارات الأحدث، يمكنك استخدام حلقة for القائمة على النطاق، كما هو موضح أدناه. عند اكتمال الكود، سيحتوي الإجمالي على مجموع عناصر مصفوفة الوحدات.

```
int total = 0; // Initialize accumulator  
for (int val : units)  
    total += val;
```

Getting the Average of the Values in a Numeric Array

الخطوة الأولى لحساب متوسط جميع قيم المصفوفة هي جمعها. الخطوة الثانية هي قسمة المجموع على عدد عناصر المصفوفة. افترض أن العبارات التالية تظهر في برنامج، وأن القيم مخزنة في مصفوفة الدرجات:

```
const int NUM_SCORES = 10;  
double scores[NUM_SCORES];
```

يحسب الكود التالي متوسط القيم في مصفوفة النتائج. عند اكتمال الكود، يُخزن المتوسط في متغير المتوسط.

```
double total = 0; // Initialize accumulator  
double average; // Will hold the average  
for (int count = 0; count < NUM_SCORES; count++)  
    total += scores[count];  
average = total / NUM_SCORES;
```

في C++ 11 والإصدارات الأحدث، يمكنك استخدام حلقة for القائمة على النطاق، كما هو موضح أدناه. عند اكتمال الكود، سيتم تخزين المتوسط في متغير المتوسط.

```
double total = 0; // Initialize accumulator
double average; // Will hold the average
for (int val : scores)
    total += val;
average = total / NUM_SCORES;
```

Finding the Highest and Lowest Values in a Numeric Array

تتشابه خوارزميات إيجاد أعلى وأدنى قيمة في المصفوفة إلى حد كبير. لنلق نظرة أولاً على الكود المستخدم لإيجاد أعلى قيمة في المصفوفة. لنفترض أن الكود التالي موجود في برنامج، وأن القيم مُخزّنة مسبقاً في مصفوفة الأرقام:

```
const int SIZE = 50;
int numbers[SIZE];
```

الكود للعثور على أعلى قيمة في المصفوفة هو كما يلي:

```
int count;
int highest;
highest = numbers[0];
for (count = 1; count < SIZE; count++)
{
    if (numbers[count] > highest)
        highest = numbers[count];
}
```

الكود التالي يبحث عن أقل قيمة في المصفوفة. كما ترى، فهو مطابق تقريباً لكود إيجاد أعلى قيمة.

```
int count;
int lowest;
lowest = numbers[0];
for (count = 1; count < SIZE; count++)
{
    if (numbers[count] < lowest)
        lowest = numbers[count];
}
```

Partially Filled Arrays

أحياناً تحتاج إلى تخزين سلسلة من العناصر في مصفوفة، ولكنك لا تعرف عددها. ونتيجةً لذلك، لا تعرف العدد الدقيق للعناصر اللازمة للمصفوفة. أحد الحلول هو جعل المصفوفة كبيرة بما يكفي لاستيعاب أكبر عدد ممكن من العناصر. ومع ذلك، قد يؤدي هذا إلى مشكلة أخرى. إذا كان العدد الفعلي للعناصر المخزنة في المصفوفة أقل من عدد العناصر، فسيتم ملء المصفوفة جزئياً فقط. عند معالجة مصفوفة ممتلئة جزئياً، يجب معالجة العناصر التي تحتوي على عناصر بيانات صالحة فقط.

عادةً ما تُستخدم المصفوفة الممتلئة جزئياً مع متغير عدد صحيح مصاحب يحمل عدد العناصر المخزنة فيها. على سبيل المثال، لنفترض أن برنامجاً يستخدم الكود التالي لإنشاء مصفوفة تحتوي على 100 عنصر، ومتغير عدد صحيح يُسمى count يحمل عدد العناصر المخزنة فيها.

```
const int SIZE = 100;
int numbers[SIZE];
int count = 0;
```

في كل مرة نضيف عنصراً إلى المصفوفة، يجب زيادة قيمة count. يوضح الكود التالي ذلك:

```
int num;
cout << "Enter a number or -1 to quit: ";
cin >> num;
while (num != -1 && count < SIZE)
{
    count++;
    numbers[count - 1] = num;
    cout << "Enter a number or -1 to quit: ";
    cin >> num;
}
```

كل تكرار لهذه الحلقة التي يتحكم بها الحارس (sentinel-controlled loop) يسمح للمستخدم بإدخال رقم لتخزينه في المصفوفة، أو -1 للخروج. يُزاد متغير count ثم يُستخدم لحساب قيمة العنصر التالي المتاح في المصفوفة. عندما يُدخل المستخدم -1، أو يتجاوز count 99، تتوقف الحلقة. يعرض الكود التالي جميع العناصر الصالحة في المصفوفة:

```
for (int index = 0; index < count; index++)
{
    cout << numbers[index] << endl;
}
```

يوضح البرنامج 7-14 كيفية استخدام هذه التقنية لقراءة عدد غير معروف من العناصر من ملف إلى مصفوفة. يقرأ البرنامج القيم من الملف numbers.txt.

Program 7-14

```
1 //This program reads data from a file into an array.
2 #include <iostream>
3 #include <fstream>
4 using namespace std;
5
6 int main()
7 {
8     const int ARRAY_SIZE = 100; // Array size
9     int numbers[ARRAY_SIZE];    // Array with 100 elements
```

```

10     int count = 0;                // Loop counter variable
11     ifstream inputFile;           // Input file stream object
12
13     inputFile.open("numbers.txt"); // Open the file.
14
15     // Read the numbers from the file into the array.
16     // After this loop executes, the count variable will hold
17     // the number of values that were stored in the array.
18     while (count < ARRAY_SIZE && inputFile >> numbers[count])
19         count++;
20
21     // Close the file.
22     inputFile.close();
23
24     // Display the numbers read.
25     cout << "The numbers are: ";
26     for (int index = 0; index < count; index++)
27         cout << numbers[index] << " ";
28     cout << endl;
29     return 0;
30 }

```

Program Output

The numbers are: 47 89 65 36 12 25 17 8 62 10 87 62

Comparing Arrays

لقد لاحظنا سابقاً أنه لا يمكنك ببساطة إسناد مصفوفة إلى أخرى. يجب عليك إسناد كل عنصر من المصفوفة الأولى إلى عنصر من المصفوفة الثانية. بالإضافة إلى ذلك، لا يمكنك استخدام عامل == مع أسماء مصفوفتين لتحديد ما إذا كانتا متساويتين. على سبيل المثال، يبدو أن الكود التالي يُقارن مصفوفتين، ولكنه في الواقع لا يفعل ذلك:

```

int firstArray[] = { 5, 10, 15, 20, 25 };
int secondArray[] = { 5, 10, 15, 20, 25 };
if (firstArray == secondArray) // This is a mistake.
    cout << "The arrays are the same.\n";
else
    cout << "The arrays are not the same.\n";

```

عند استخدام عامل == مع أسماء المصفوفات، يقارن هذا العامل عناوين الذاكرة الأولية للمصفوفات، وليس محتوياتها. من الواضح أن اسمي المصفوفتين في هذا الكود يحملان عناوين ذاكرة مختلفة. لذلك، تكون نتيجة التعبير `firstArray == secondArray` خاطئة، ويُشير الكود إلى أن المصفوفات ليست متطابقة.

لمقارنة محتويات مصفوفتين، يجب مقارنة عناصرهما. على سبيل المثال، انظر الكود التالي:

```
const int SIZE = 5;
int firstArray[SIZE] = { 5, 10, 15, 20, 25 };
int secondArray[SIZE] = { 5, 10, 15, 20, 25 };
bool arraysEqual = true; // Flag variable
int count = 0; // Loop counter variable
// Determine whether the elements contain the same data.
while (arraysEqual && count < SIZE)
{
    if (firstArray[count] != secondArray[count])
        arraysEqual = false;
    count++;
}

if (arraysEqual)
    cout << "The arrays are equal.\n";
else
    cout << "The arrays are not equal.\n";
```

Using Structured Binding Declarations with Arrays

في بعض الحالات، قد تحتاج إلى استرداد القيم الفردية المخزنة في مصفوفة وتعيينها لمتغيرات عادية. في إصدار C++ 17، تتيح لك ميزة جديدة تُعرف باسم إعلانات الربط الهيكلي (*structured binding declarations*) كتابة جملة واحدة تُعرّف مجموعة من المتغيرات وتُهيئ هذه المتغيرات بالقيم المخزنة في مصفوفة. تُعرف هذه العملية باسم فك ضغط المصفوفة (*unpacking*). فيما يلي الصيغة العامة لإعلان الربط الهيكلي:

```
auto [variable1, variable2, etc...] = array;
```

سيتم تعيين قيمة العنصر 0 للمتغير الأول المدرج بين القوسين، وسيتم تعيين قيمة العنصر 1 للمتغير الثاني المدرج بين القوسين، وهكذا.

يجب أن يكون عدد المتغيرات المدرجة بين القوسين المربعين مساوياً لعدد عناصر المصفوفة. وإلا، فسيحدث خطأ في المترجم.

لنلق نظرة على مثال. لنفترض أن لدينا ثلاث قيم مخزنة في مصفوفة، كما هو موضح هنا:

```
int testScores[] = {80, 90, 100};
```

العبرة التالية هي إعلان ربط منظم يقوم بفك ضغط المصفوفة وتعيين قيمها للمتغيرات:

```
auto [score1, score2, score3] = testScores;
```

7.5 Focus on Software Engineering: Using Parallel Arrays

المفهوم: باستخدام نفس الرمز السفلي (subscript)، يمكنك بناء علاقات بين البيانات المخزنة في مصفوفتين أو أكثر.

أحيانًا يكون من المفيد تخزين البيانات ذات الصلة في مصفوفتين أو أكثر. ويزداد هذا الأمر فائدة خاصة عندما تكون البيانات ذات الصلة من أنواع مختلفة. على سبيل المثال، يُعد البرنامج 7-15 نسخة أخرى من برنامج الرواتب. يستخدم مصفوفتين: واحدة لتخزين ساعات عمل كل موظف (كأعداد صحيحة)، والأخرى لتخزين أجر كل موظف بالساعة (كأعداد مضاعفة).

Program 7-15

```
1 // This program uses two parallel arrays: one for hours
2 // worked and one for pay rate.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     const int NUM_EMPLOYEES = 5; // Number of employees
10    int hours[NUM_EMPLOYEES];     // Holds hours worked
11    double payRate[NUM_EMPLOYEES]; // Holds pay rates
12
```

Program 7-15 (continued)

```
13 // Input the hours worked and the hourly pay rate.
14 cout << "Enter the hours worked by " << NUM_EMPLOYEES
15     << " employees and their\n"
16     << "hourly pay rates.\n";
17 for (int index = 0; index < NUM_EMPLOYEES; index++)
18 {
19     cout << "Hours worked by employee #" << (index+1) << ": ";
20     cin >> hours[index];
21     cout << "Hourly pay rate for employee #" << (index+1) << ": ";
22     cin >> payRate[index];
23 }
24
25 // Display each employee's gross pay.
26 cout << "Here is the gross pay for each employee:\n";
27 cout << fixed << showpoint << setprecision(2);
28 for (int index = 0; index < NUM_EMPLOYEES; index++)
29 {
30     double grossPay = hours[index] * payRate[index];
31     cout << "Employee #" << (index + 1);
32     cout << ": $" << grossPay << endl;
33 }
34 return 0;
35 }
```

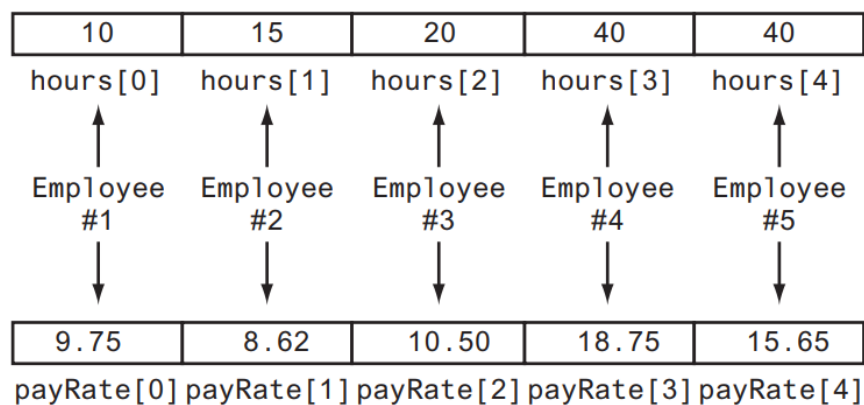
Program Output with Example Input Shown in Bold

Enter the hours worked by 5 employees and their hourly pay rates.

Hours worked by employee #1: **10**
Hourly pay rate for employee #1: **9.75**
Hours worked by employee #2: **15**
Hourly pay rate for employee #2: **8.62**
Hours worked by employee #3: **20**
Hourly pay rate for employee #3: **10.50**
Hours worked by employee #4: **40**
Hourly pay rate for employee #4: **18.75**
Hours worked by employee #5: **40**
Hourly pay rate for employee #5: **15.65**
Here is the gross pay for each employee:
Employee #1: \$97.50
Employee #2: \$129.30
Employee #3: \$210.00
Employee #4: \$750.00
Employee #5: \$626.00

This concept is illustrated in Figure 7-13.

Figure 7-13 Parallel arrays



7.6 Arrays as Function Arguments

المفهوم: لتمرير مصفوفة كمعامل إلى دالة، مرّر اسم المصفوفة.

في كثير من الأحيان، ستحتاج إلى كتابة دوال لمعالجة البيانات في المصفوفات. على سبيل المثال، يمكن كتابة دوال لوضع قيم في مصفوفة، أو عرض محتوياتها على الشاشة، أو جمع عناصر المصفوفة، أو حساب متوسطها. عادةً، تقبل هذه الدوال المصفوفة كمعامل.

عند تمرير عنصر واحد من مصفوفة إلى دالة، يتم التعامل معه كأبي متغير آخر. على سبيل المثال، يعرض البرنامج 7-16 حلقة تكرار تُمرّر عنصرًا واحدًا من أرقام المصفوفة إلى الدالة showValue في كل مرة تتكرر فيها الحلقة.

Program 7-16

```
1 // This program demonstrates that an array element is passed
2 // to a function like any other variable.
3 #include <iostream>
4 using namespace std;
5
6 void showValue(int); // Function prototype
7
8 int main()
9 {
10     const int SIZE = 8;
11     int numbers[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
12
13     for (int index = 0; index < SIZE; index++)
14         showValue(numbers[index]);
15     return 0;
16 }
17
18 //*****
19 // Definition of function showValue. *
20 // This function accepts an integer argument. *
21 // The value of the argument is displayed. *
22 //*****
23
24 void showValue(int num)
25 {
26     cout << num << " ";
27 }
```

Program Output

5 10 15 20 25 30 35 40

إذا كُتبت الدالة لقبول المصفوفة بأكملها كمعامل، فسيتم إعداد المُعامل بشكل مختلف. في تعريف الدالة التالي، يتبع المُعامل nums مجموعة فارغة من الأقواس. هذا يُشير إلى أن المُعامل سيكون مصفوفة، وليس قيمة واحدة.

```
void showValues(int nums[], int size)
{
    for (int index = 0; index < size; index++)
        cout << nums[index] << " ";
    cout << endl;
}
```

السبب في عدم وجود مُعلن حجم داخل أقواس nums هو أن nums ليست مصفوفة في الواقع. إنها متغير خاص يمكنه قبول عنوان المصفوفة. عند تمرير مصفوفة كاملة إلى دالة، لا يتم تمريرها بالقيمة، بل بالمرجع. تخيل وقت وحدة المعالجة المركزية والذاكرة اللازمين لإنشاء نسخة من مصفوفة مكونة من 10,000 عنصر في كل مرة يتم تمريرها إلى دالة! بدلاً من ذلك، يتم تمرير عنوان ذاكرة البداية للمصفوفة فقط. يعرض البرنامج 7-17 الدالة showValues قيد الاستخدام.

Program 7-17

```
1 // This program demonstrates an array being passed to a function.
2 #include <iostream>
3 using namespace std;
4
5 void showValues(int [], int); // Function prototype
6
7 int main()
8 {
9     const int ARRAY_SIZE = 8;
```

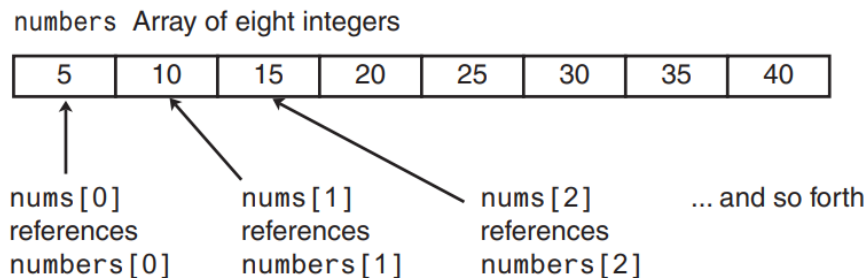
Program 7-17 (continued)

```
10     int numbers[ARRAY_SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
11
12     showValues(numbers, ARRAY_SIZE);
13     return 0;
14 }
15
16 //*****
17 // Definition of function showValue. *
18 // This function accepts an array of integers and *
19 // the array's size as its arguments. The contents *
20 // of the array are displayed. *
21 //*****
22
23 void showValues(int nums[], int size)
24 {
25     for (int index = 0; index < size; index++)
26         cout << nums[index] << " ";
27     cout << endl;
28 }
```

Program Output

5 10 15 20 25 30 35 40

Figure 7-14 Relationship between the numbers array and the nums parameter



Using const Array Parameters

أحياناً ترغب في أن تتمكن دالة من تعديل محتويات مصفوفة مُمرّرة إليها كعامل، وأحياناً لا ترغب في ذلك. يمكنك منع دالة من إجراء تغييرات على عامل مصفوفة باستخدام الكلمة المفتاحية `const` في إعلان المعلمة.

فيما يلي مثال على دالة `showValues`، الموضحة سابقاً، بعد إعادة كتابتها باستخدام معلمة `const` `array`:

```
void showValues(const int nums[], int size)
{
    for (int index = 0; index < size; index++)
        cout << nums[index] << " ";
    cout << endl;
}
```

عند تعريف مُعامل مصفوفة على أنه ثابت، لا يُسمح للدالة بإجراء تغييرات على محتويات المصفوفة. إذا حاولت جملة في الدالة تعديل المصفوفة، فسيحدث خطأ أثناء عملية التجميع (Compiling). كإجراء احترازي، يجب عليك دائماً استخدام مُعاملات المصفوفة الثابتة في أي دالة غير مُخصصة لتعديل مُعامل المصفوفة. بهذه الطريقة، ستفشل الدالة في التجميع (compile) إذا كتبت فيها عن غير قصد شيفرة تُعدّل المصفوفة.

7.7 Two-Dimensional Arrays

المفهوم: المصفوفة ثنائية الأبعاد تشبه عدة مصفوفات متطابقة مُجمّعة معاً. وهي مفيدة لتخزين مجموعات متعددة من البيانات.

المصفوفات التي درستها حتى الآن هي مصفوفات أحادية البعد. تُسمى أحادية البعد لأنها لا تنتسج إلا لمجموعة بيانات واحدة. أما المصفوفات ثنائية الأبعاد، والتي تُسمى أحياناً مصفوفات ثنائية الأبعاد (2D arrays)، فتنتسج لمجموعات بيانات متعددة. من الأفضل تصور المصفوفة ثنائية الأبعاد على أنها تتكون من صفوف وأعمدة من العناصر، كما هو موضح في الشكل 7-15.

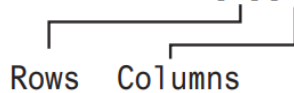
يوضح هذا الشكل مصفوفة لنتائج الاختبارات، تتكون من ثلاثة صفوف وأربعة أعمدة.

Figure 7-15 Two-dimensional array

	Column 0	Column 1	Column 2	Column 3
Row 0	scores[0] [0]	scores[0] [1]	scores[0] [2]	scores[0] [3]
Row 1	scores[1] [0]	scores[1] [1]	scores[1] [2]	scores[1] [3]
Row 2	scores[2] [0]	scores[2] [1]	scores[2] [2]	scores[2] [3]

لتعريف مصفوفة ثنائية الأبعاد، يلزم وجود مُعلّني حجم. الأول لعدد الصفوف، والثاني لعدد الأعمدة. فيما يلي مثال لتعريف مصفوفة ثنائية الأبعاد بثلاثة صفوف وأربعة أعمدة:

```
double scores[3][4];
```



عادةً ما تستخدم البرامج التي تمر عبر كل عنصر من عناصر مصفوفة ثنائية الأبعاد حلقات متداخلة. البرنامج 7-21 مثال على ذلك.

Program 7-21

```
1 // This program demonstrates a two-dimensional array.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 int main()
7 {
8     const int NUM_DIVS = 3;           // Number of divisions
9     const int NUM_QTRS = 4;           // Number of quarters
10    double sales[NUM_DIVS][NUM_QTRS]; // Array with 3 rows and 4 columns.
11    double totalSales = 0;             // To hold the total sales.
12    int div, qtr;                      // Loop counters.
13
14    cout << "This program will calculate the total sales of\n";
15    cout << "all the company's divisions.\n";
16    cout << "Enter the following sales information:\n\n";
17
18    // Nested loops to fill the array with quarterly
19    // sales figures for each division.
20    for (div = 0; div < NUM_DIVS; div++)
21    {
22        for (qtr = 0; qtr < NUM_QTRS; qtr++)
23        {
24            cout << "Division " << (div + 1);
25            cout << ", Quarter " << (qtr + 1) << ": $";
26            cin >> sales[div][qtr];
27        }
28        cout << endl; // Print blank line.
29    }
30
31    // Nested loops used to add all the elements.
32    for (div = 0; div < NUM_DIVS; div++)
33    {
34        for (qtr = 0; qtr < NUM_QTRS; qtr++)
35            totalSales += sales[div][qtr];
36    }
37
38    cout << fixed << showpoint << setprecision(2);
39    cout << "The total sales for the company are: $";
40    cout << totalSales << endl;
41    return 0;
42 }
```

(program continues)

Program 7-21

(continued)

Program Output with Example Input Shown in Bold

This program will calculate the total sales of all the company's divisions.

Enter the following sales data:

Division 1, Quarter 1: **\$31569.45**

Division 1, Quarter 2: **\$29654.23**

Division 1, Quarter 3: **\$32982.54**

Division 1, Quarter 4: **\$39651.21**

Division 2, Quarter 1: **\$56321.02**

Division 2, Quarter 2: **\$54128.63**

Division 2, Quarter 3: **\$41235.85**

Division 2, Quarter 4: **\$54652.33**

Division 3, Quarter 1: **\$29654.35**

Division 3, Quarter 2: **\$28963.32**

Division 3, Quarter 3: **\$25353.55**

Division 3, Quarter 4: **\$32615.88**

The total sales for the company are: \$456782.34

<code>int hours[3][2] = {{8, 5}, {7, 9}, {6, 3}};</code>	Good way
<code>int hours[3][2] = {8, 5, 7, 9, 6, 3};</code>	Bad way

Figure 7-16 Two-dimensional array initialization

	Column 0	Column 1
Row 0	8	5
Row 1	7	9
Row 2	6	3

<code>int table[3][2] = {{1}, {3, 4}, {5}};</code>
table[0][0] is initialized to 1, table[1][0] is initialized to 3, table[1][1] is initialized to 4, and table[2][0] is initialized to 5. table[0][1] and table[2][1] are not initialized. Because some of the array elements are initialized, these two initialized elements are automatically set to zero.

Passing Two-Dimensional Arrays to Functions

يوضح البرنامج 22-7 تمرير مصفوفة ثنائية الأبعاد إلى دالة. عند تمرير مصفوفة ثنائية الأبعاد إلى دالة، يجب أن يحتوي نوع المعلمة على مُعلن حجم لعدد الأعمدة. فيما يلي رأس الدالة showArray، من البرنامج 22-7:

void showArray(const int numbers[][COLS], int rows)

COLS هو ثابت عام مُسمّى، مُعيّن على القيمة 4. يمكن للدالة قبول أي مصفوفة أعداد صحيحة ثنائية الأبعاد، طالما أنها تتكون من أربعة أعمدة. في البرنامج، تعرض الدالة محتويات مصفوفتين منفصلتين.

Program 7-22

```
1 // This program demonstrates accepting a 2D array argument.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 // Global constants
7 const int COLS = 4; // Number of columns in each array
8 const int TBL1_ROWS = 3; // Number of rows in table1
9 const int TBL2_ROWS = 4; // Number of rows in table2
10
11 void showArray(const int[][COLS], int); // Function prototype
12
13 int main()
14 {
15     int table1[TBL1_ROWS][COLS] = {{1, 2, 3, 4},
16                                     {5, 6, 7, 8},
17                                     {9, 10, 11, 12}};
18     int table2[TBL2_ROWS][COLS] = {{10, 20, 30, 40},
19                                    {50, 60, 70, 80},
20                                    {90, 100, 110, 120},
21                                    {130, 140, 150, 160}};
```

Program 7-22 (continued)

```
22
23     cout << "The contents of table1 are:\n";
24     showArray(table1, TBL1_ROWS);
25     cout << "The contents of table2 are:\n";
26     showArray(table2, TBL2_ROWS);
27     return 0;
28 }
29
30 //*****
31 // Function Definition for showArray *
32 // The first argument is a two-dimensional int array with COLS *
33 // columns. The second argument, rows, specifies the number of *
34 // rows in the array. The function displays the array's contents. *
35 //*****
36
37 void showArray(const int numbers[][COLS], int rows)
38 {
39     for (int x = 0; x < rows; x++)
40     {
41         for (int y = 0; y < COLS; y++)
42         {
43             cout << setw(4) << numbers[x][y] << " ";
44         }
45         cout << endl;
46     }
47 }
```

Program Output

```
The contents of table1 are:
1  2  3  4
5  6  7  8
9 10 11 12
The contents of table2 are:
10 20 30 40
50 60 70 80
90 100 110 120
130 140 150 160
```

تتطلب لغة C++ تحديد الأعمدة في نموذج الدالة ورأسها نظراً لطريقة تخزين المصفوفات ثنائية الأبعاد في الذاكرة. يتلو كل صف صفًا، كما هو موضح في الشكل 7-17.

Figure 7-17 Memory organization of a two-dimensional array



عندما يُنشئ المُجمِّع (compiler) شيفرةً (code) للوصول إلى عناصر مصفوفة ثنائية الأبعاد، فإنه يحتاج إلى معرفة عدد البايتات التي تفصل الصفوف في الذاكرة. ويُعدّ عدد الأعمدة عاملاً حاسماً في هذا الحساب.

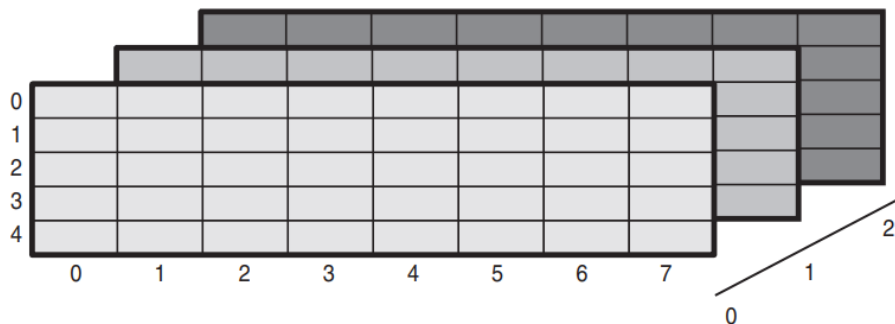
7.8 Arrays with Three or More Dimensions

المفهوم: لا تُحدّ لغة C++ عدد الأبعاد التي يمكن أن تحتويها المصفوفة. من الممكن إنشاء مصفوفات بأبعاد متعددة، لنمذجة البيانات التي تظهر في مجموعات متعددة.

تتيح لك لغة C++ إنشاء مصفوفات بأي عدد من الأبعاد تقريباً. إليك مثال على تعريف مصفوفة ثلاثية الأبعاد:

```
double seats[3][5][8];
```

Figure 7-18 A three-dimensional array



NOTE: When writing functions that accept multi-dimensional arrays as arguments, all but the first dimension must be explicitly stated in the parameter list.

7.9 Introduction to the STL (Standard Template Library) vector

المفهوم: تُقدم مكتبة القوالب القياسية نوع بيانات متجه، وهو يتفوق في نواحٍ عديدة على المصفوفات القياسية.

مكتبة القوالب القياسية (STL) هي مجموعة من أنواع البيانات والخوارزميات التي يمكنك استخدامها في براملك. هذه الأنواع من البيانات والخوارزميات مُعرّفة من قِبل المبرمج. وهي ليست جزءاً من لغة C++، بل أنشئت بالإضافة إلى أنواع البيانات المُدمجة. إذا كنت تُخطط لمواصلة دراستك في مجال علوم الحاسوب، فينبغي عليك أن تُصبح مُلمّاً بمكتبة القوالب القياسية.

في هذا القسم، سنتعلم استخدام نوع بيانات المتجه (Vector). المتجه هو حاوية لتخزين البيانات. يشبه المصفوفة في النواحي التالية:

- يحمل المتجه سلسلة من القيم أو العناصر.
 - يخزن المتجه عناصره في مواقع ذاكرة متجاورة.
 - يمكنك استخدام مُعامل المصفوفة السفلي [] لقراءة العناصر الفردية في المتجه.
- ومع ذلك، يوفر المتجه مزايا عديدة مقارنةً بالمصفوفات. إليك بعضاً منها:
- ليس عليك تحديد عدد العناصر التي سيحتويها المتجه.
 - إذا أضفت قيمة إلى متجه ممتلئ بالفعل، فسيزيد المتجه حجمه تلقائياً لاستيعاب القيمة الجديدة.
 - يمكن للمتجهات الإبلاغ عن عدد العناصر التي تحتويها.



NOTE: To use the vector data type, you should also have the `using namespace std;` statement in your program.

Defining a vector

لاستخدام المتجهات في برنامجك، يجب عليك تضمين ملف رأس `<vector>` مع العبارة التالية:

```
#include <vector>
```

تختلف صياغة تعريف المتجه إلى حد ما عن صياغة تعريف متغير عادي أو مصفوفة. إليك مثال:

```
vector<int> numbers;
```

تُعرف هذه العبارة `numbers` كمتجه من الأعداد الصحيحة. لاحظ أن نوع البيانات مُحاط بأقواس زاوية، مباشرةً بعد كلمة `vector`. ولأن حجم المتجه يتوسع عند إضافة قيم إليه، فلا حاجة لإعلان حجم. يمكنك تحديد حجم ابتدائي، إن شئت. إليك مثال:

```
vector<int> numbers(10);
```



NOTE: If you specify a starting size for a vector, the size declarator is enclosed in parentheses, not in square brackets.

عند تحديد حجم ابتدائي لمتجه، يمكنك أيضاً تحديد قيمة تهيئة. تُنسخ قيمة التهيئة إلى كل عنصر. إليك مثال:

```
vector<int> numbers(10, 2);
```

في هذه العبارة، تُعرف `numbers` كمتجه من 10 أعداد صحيحة. كل عنصر في الأرقام مُهيأ بالقيمة 2.

يمكنك أيضاً تهيئة متجه بقيم متجه آخر. على سبيل المثال، انظر إلى العبارة التالية: افترض أن `set1` متجه من الأعداد الصحيحة يحتوي على قيم مخزنة فيه.

```
vector<int> set2(set1);
```

بعد تنفيذ هذه العبارة، ستكون `set2` نسخة من المجموعة `set1`.

Table 7-3 vector Definitions

Definition Format	Description
<code>vector<float> amounts;</code>	Defines <code>amounts</code> as an empty vector of floats.
<code>vector<string> names;</code>	Defines <code>names</code> as an empty vector of string objects.
<code>vector<int> scores(15);</code>	Defines <code>scores</code> as a vector of 15 ints.
<code>vector<char> letters(25, 'A');</code>	Defines <code>letters</code> as a vector of 25 characters. Each element is initialized with 'A'.
<code>vector<double> values2(values1);</code>	Defines <code>values2</code> as a vector of doubles. All the elements of <code>values1</code> , which is also a vector of doubles, are copied to <code>value2</code> .

Using an Initialization List with a vector

إذا كنت تستخدم C++ 11 أو إصدارًا أحدث، فيمكنك تهيئة متجه بقائمة من القيم، كما هو موضح في هذا المثال:

```
vector<int> numbers { 10, 20, 30, 40 };
```

تُعرّف هذه العبارة متجهًا من الأعداد الصحيحة (ints) باسم `numbers`. يتكون المتجه من أربعة عناصر، مُهيأة بالقيم 10، 20، 30، و40. لاحظ أن قائمة التهيئة مُحاطة بمجموعة من الأقواس، ولكن لا يُستخدم عامل = قبلها.

Storing and Retrieving Values in a vector

لتخزين قيمة في عنصر موجود بالفعل في متجه، يمكنك استخدام عامل المصفوفة السفلي []. على سبيل المثال، انظر البرنامج 7-24.

Program 7-24

```

1  // This program stores, in two vectors, the hours worked by 5
2  // employees, and their hourly pay rates.
3  #include <iostream>
4  #include <iomanip>
5  #include <vector>
6  using namespace std;
7
8  int main()
9  {
10     const int NUM_EMPLOYEES = 5;           // Number of employees
11     vector<int> hours(NUM_EMPLOYEES);      // A vector of integers
12     vector<double> payRate(NUM_EMPLOYEES); // A vector of doubles
13     int index;                             // Loop counter
14
15     // Input the data.
16     cout << "Enter the hours worked by " << NUM_EMPLOYEES;
17     cout << " employees and their hourly rates.\n";

```

```

18     for (index = 0; index < NUM_EMPLOYEES; index++)
19     {
20         cout << "Hours worked by employee #" << (index + 1);
21         cout << ": ";
22         cin >> hours[index];
23         cout << "Hourly pay rate for employee #";
24         cout << (index + 1) << ": ";
25         cin >> payRate[index];
26     }
27
28     // Display each employee's gross pay.
29     cout << "\nHere is the gross pay for each employee:\n";
30     cout << fixed << showpoint << setprecision(2);
31     for (index = 0; index < NUM_EMPLOYEES; index++)
32     {
33         double grossPay = hours[index] * payRate[index];
34         cout << "Employee #" << (index + 1);
35         cout << ": $" << grossPay << endl;
36     }
37     return 0;
38 }

```

Program Output with Example Input Shown in Bold

Enter the hours worked by 5 employees and their hourly rates.

Hours worked by employee #1: **10**

Hourly pay rate for employee #1: **9.75**

Hours worked by employee #2: **15**

Hourly pay rate for employee #2: **8.62**

Hours worked by employee #3: **20**

Hourly pay rate for employee #3: **10.50**

Hours worked by employee #4: **40**

Hourly pay rate for employee #4: **18.75**

Hours worked by employee #5: **40**

Hourly pay rate for employee #5: **15.65**

Here is the gross pay for each employee:

Employee #1: \$97.50

Employee #2: \$129.30

Employee #3: \$210.00

Employee #4: \$750.00

Employee #5: \$626.00

Using the Range-Based for Loop with a vector

مع إصدار C++ 11 والإصدارات الأحدث، يمكنك استخدام حلقة for القائمة على النطاق للانتقال عبر عناصر المتجه، كما هو موضح في البرنامج 7-25.

Program 7-25

```

1  // This program demonstrates the range-based for loop with a vector.
2  include <iostream>
3  #include <vector>
4  using namespace std;
5
6  int main()
7  {
8      // Define and initialize a vector.
9      vector<int> numbers { 10, 20, 30, 40, 50 };
10
11     // Display the vector elements.
12     for (int val : numbers)
13         cout << val << endl;
14
15     return 0;
16 }

```

Program Output

10
20
30
40
50

يوضح البرنامج 7-26 كيفية استخدام متغير مرجعي مع حلقة for المستندة إلى النطاق لتخزين العناصر في متجه.

Program 7-26

```
1 // This program demonstrates the range-based for loop with a vector.
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     // Define a vector.
9     vector<int> numbers(5);
10
11     // Get values for the vector elements.
12     for (int &val : numbers)
13     {
14         cout << "Enter an integer value: ";
15         cin >> val;
16     }
17
18     // Display the vector elements.
19     cout << "Here are the values you entered:\n";
20     for (int val : numbers)
21         cout << val << endl;
22
23     return 0;
24 }
```

Program Output with Example Input Shown in Bold

```
Enter an integer value: 1 
Enter an integer value: 2 
Enter an integer value: 3 
Enter an integer value: 4 
Enter an integer value: 5 
Here are the values you entered:
1
2
3
4
5
```

Using the push_back Member Function

لا يمكنك استخدام عامل التشغيل [] للوصول إلى عنصر متجه غير موجود. لتخزين قيمة في متجه ليس له حجم ابتدائي، أو ممتلئ بالفعل، استخدم دالة العضو push_back. تقبل دالة العضو push_back قيمة كمعامل، وتخزنها بعد آخر عنصر في المتجه. (تدفع القيمة إلى الجزء الخلفي من المتجه). إليك مثال:

numbers.push_back(25);

بافتراض أن numbers متجه من الأعداد الصحيحة، تُخزن هذه العبارة 25 كعنصر أخير. إذا كانت الأعداد ممتلئة، تُنشئ العبارة عنصرًا أخيرًا جديدًا وتُخزن 25 فيه. إذا لم تكن هناك أي عناصر في الأعداد، تُنشئ هذه العبارة عنصرًا وتُخزن 25 فيه.

البرنامج 27-7 هو تعديل للبرنامج 24-7. مع ذلك، يتيح هذا الإصدار للمستخدم تحديد عدد الموظفين. يُعرّف المتجهان، الساعات ومعدل الأجر، بدون أحجام ابتدائية. ولأن هذين المتجهين لا يحتويان على عناصر ابتدائية، تُستخدم دالة العضو `push_back` لتخزين القيم فيهما.

Program 7-27

```

1 // This program stores, in two arrays, the hours worked by 5
2 // employees, and their hourly pay rates.
3 #include <iostream>
4 #include <iomanip>
5 #include <vector>
6 using namespace std;
7
8 int main()
9 {
10     vector<int> hours;           // hours is an empty vector
11     vector<double> payRate;     // payRate is an empty vector
12     int numEmployees;          // The number of employees
13     int index;                 // Loop counter
14
15     // Get the number of employees.
16     cout << "How many employees do you have? ";
17     cin >> numEmployees;
18
19     // Input the payroll data.
20     cout << "Enter the hours worked by " << numEmployees;
21     cout << " employees and their hourly rates.\n";
22     for (index = 0; index < numEmployees; index++)
23     {
24         int tempHours;          // To hold the number of hours entered
25         double tempRate;       // To hold the pay rate entered
26
27         cout << "Hours worked by employee #" << (index + 1);
28         cout << ": ";
29         cin >> tempHours;
30         hours.push_back(tempHours); // Add an element to hours
31         cout << "Hourly pay rate for employee #";
32         cout << (index + 1) << ": ";
33         cin >> tempRate;
34         payRate.push_back(tempRate); // Add an element to payRate
35     }
36
37     // Display each employee's gross pay.
38     cout << "Here is the gross pay for each employee:\n";
39     cout << fixed << showpoint << setprecision(2);
40     for (index = 0; index < numEmployees; index++)
41     {
42         double grossPay = hours[index] * payRate[index];
43         cout << "Employee #" << (index + 1);
44         cout << ": $" << grossPay << endl;
45     }
46     return 0;
47 }

```

Program Output with Example Input Shown in Bold

```

How many employees do you have? 3  Enter
Enter the hours worked by 3 employees and their hourly rates.
Hours worked by employee #1: 40  Enter
Hourly pay rate for employee #1: 12.63  Enter
Hours worked by employee #2: 25  Enter
Hourly pay rate for employee #2: 10.35  Enter
Hours worked by employee #3: 45  Enter
Hourly pay rate for employee #3: 22.65  Enter

Here is the gross pay for each employee:
Employee #1: $505.20
Employee #2: $258.75
Employee #3: $1019.2

```

Determining the Size of a vector

لأن المتجه قادر على تحديد حجمه، فلا حاجة لهذه الدالة لقبول وسيطة ثانية تُشير إلى عدد عناصر المتجه. يوضح البرنامج 28-7 هذه الدالة.

Program 7-28

```
1 // This program demonstrates the vector size
2 // member function.
3 #include <iostream>
4 #include <vector>
5 using namespace std;
6
7 // Function prototype
8 void showValues(vector<int>);
9
10 int main()
11 {
12     vector<int> values;
13
14     // Put a series of numbers in the vector.
15     for (int count = 0; count < 7; count++)
16         values.push_back(count * 2);
17
18     // Display the numbers.
19     showValues(values);
20     return 0;
21 }
22
23 //*****
24 // Definition of function showValues. *
25 // This function accepts an int vector as its *
26 // argument. The value of each of the vector's *
27 // elements is displayed. *
28 //*****
29
30 void showValues(vector<int> vect)
31 {
32     for (int count = 0; count < vect.size(); count++)
33         cout << vect[count] << endl;
34 }
```

Program Output

```
0
2
4
6
8
10
12
```

Removing Elements from a vector

يوضح البرنامج 7-29 الوظيفة.

Program 7-29

```
1 // This program demonstrates the vector pop_back member function.
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     vector<int> values;
9
10    // Store values in the vector.
11    values.push_back(1);
12    values.push_back(2);
13    values.push_back(3);
14    cout << "The size of values is " << values.size() << endl;
15
16    // Remove a value from the vector.
17    cout << "Popping a value from the vector ... \n";
18    values.pop_back();
19    cout << "The size of values is now " << values.size() << endl;
20
21    // Now remove another value from the vector.
22    cout << "Popping a value from the vector ... \n";
23    values.pop_back();
24    cout << "The size of values is now " << values.size() << endl;
25
26    // Remove the last value from the vector.
27    cout << "Popping a value from the vector ... \n";
28    values.pop_back();
29    cout << "The size of values is now " << values.size() << endl;
30    return 0;
31 }
```

Program Output

```
The size of values is 3
Popping a value from the vector...
The size of values is now 2
Popping a value from the vector...
The size of values is now 1
Popping a value from the vector...
The size of values is now 0
```

Clearing a vector

يوضح البرنامج 30-7 الوظيفة.

Program 7-30

```
1 // This program demonstrates the vector clear member function.
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     vector<int> values(100);
9
10    cout << "The values vector has "
11          << values.size() << " elements.\n";
12    cout << "I will call the clear member function ... \n";
13    values.clear();
14    cout << "Now, the values vector has "
15          << values.size() << " elements.\n";
16    return 0;
17 }
```

Program Output

```
The values vector has 100 elements.
I will call the clear member function...
Now, the values vector has 0 elements.
```

Detecting an Empty vector

يستخدم البرنامج 31-7 دالة تسمى avgVector، والتي توضح دالة العضو الفارغة.

Program 7-31

```
1 // This program demonstrates the vector's empty member function.
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 // Function prototype
7 double avgVector(vector<int>);
8
```

```

9  int main()
10 {
11     vector<int> values; // A vector to hold values
12     int numValues;     // The number of values
13     double average;    // To hold the average
14
15     // Get the number of values to average.
16     cout << "How many values do you wish to average? ";
17     cin >> numValues;
18
19     // Get the values and store them in the vector.
20     for (int count = 0; count < numValues; count++)
21     {
22         int tempValue;
23         cout << "Enter a value: ";
24         cin >> tempValue;
25         values.push_back(tempValue);
26     }
27
28     // Get the average of the values and display it.
29     average = avgVector(values);
30
31     cout << "Average: " << average << endl;
32     return 0;
33 }
34 //*****
35 // Definition of function avgVector. *
36 // This function accepts an int vector as its argument. If *
37 // the vector contains values, the function returns the *
38 // average of those values. Otherwise, an error message is *
39 // displayed and the function returns 0.0. *
40 //*****
41
42 double avgVector(vector<int> vect)
43 {
44     int total = 0; // accumulator
45     double avg;   // average
46
47     if (vect.empty()) // Determine if the vector is empty
48     {
49         cout << "No values to average.\n";
50         avg = 0.0;
51     }
52     else
53     {
54         for (int count = 0; count < vect.size(); count++)
55             total += vect[count];
56         avg = total / vect.size();
57     }
58     return avg;
59 }

```

Program Output with Example Input Shown in Bold

```

How many values do you wish to average? 5 
Enter a value: 12
Enter a value: 18
Enter a value: 3
Enter a value: 7
Enter a value: 9
Average: 9

```

Program Output with Different Example Input Shown in Bold

```

How many values do you wish to average? 0 
No values to average.
Average: 0

```

Table 7-4 Some of the vector Member Functions

Member Function	Description
<code>at(<i>element</i>)</code>	Returns the value of the element located at <i>element</i> in the vector. <i>Example:</i> <pre>x = vect.at(5);</pre> This statement assigns the value of the fifth element of <code>vect</code> to <code>x</code> .
<code>clear()</code>	Clears a vector of all its elements. <i>Example:</i> <pre>vect.clear();</pre> This statement removes all the elements from <code>vect</code> .
<code>empty()</code>	Returns true if the vector is empty. Otherwise, it returns false. <i>Example:</i> <pre>if (vect.empty()) cout << "The vector is empty.";</pre> This statement displays the message if <code>vect</code> is empty.
<code>pop_back()</code>	Removes the last element from the vector. <i>Example:</i> <pre>vect.pop_back();</pre> This statement removes the last element of <code>vect</code> , thus reducing its size by 1.
<code>push_back(<i>value</i>)</code>	Stores <i>value</i> in the last element of the vector. If the vector is full or empty, a new element is created. <i>Example:</i> <pre>vect.push_back(7);</pre> This statement stores 7 in the last element of <code>vect</code> .
<code>resize(<i>elements</i>, <i>value</i>)</code>	Resizes a vector by <i>elements</i> elements. Each of the new elements is initialized with the value in <i>value</i> . <i>Example:</i> <pre>vect.resize(5, 1);</pre> This statement increases the size of <code>vect</code> by five elements. The five new elements are initialized to the value 1.
<code>swap(<i>vector2</i>)</code>	Swaps the contents of the vector with the contents of <i>vector2</i> . <i>Example:</i> <pre>vect1.swap(vect2);</pre> This statement swaps the contents of <code>vect1</code> and <code>vect2</code> .

7.10 Focus on Software Engineering: Introduction to Search Algorithms

المفهوم: خوارزمية البحث هي طريقة لتحديد موقع عنصر محدد ضمن مجموعة بيانات أكبر.

يناقش هذا القسم خوارزميتين للبحث في محتويات مصفوفة. من الشائع جدًا أن تقوم البرامج ليس فقط بتخزين ومعالجة البيانات المخزنة في المصفوفات، بل أيضًا بالبحث فيها عن عناصر محددة. سيوضح لك هذا القسم طريقتين للبحث في المصفوفة (لكل منهما مزايا وعيوب):

- البحث الخطي.
- البحث الثنائي.

The Linear Search

البحث الخطي خوارزمية بسيطة للغاية. تُعرف أيضًا باسم خوارزمية البحث التسلسلي، وتستخدم حلقةً تسلسليةً للانتقال عبر مصفوفة، بدءًا من العنصر الأول. تُقارن كل عنصر بالقيمة التي يُبحث عنها، وتتوقف عند العثور على القيمة أو الوصول إلى نهاية المصفوفة. إذا لم تكن القيمة المطلوبة موجودة في المصفوفة، فستفشل الخوارزمية في البحث حتى نهاية المصفوفة.

فيما يلي شبه الكود (pseudocode) لدالة تُجري البحث الخطي:

```
Set found to false
Set position to -1
Set index to 0
While found is false and index < number of elements
    If list[index] is equal to search value
        found = true
        position = index
    End If
    Add 1 to index
End While
Return position
```

دالة البحث الخطي الموضحة أدناه هي مثال على شيفرة C++ تُستخدم لإجراء بحث خطي على مصفوفة أعداد صحيحة. يتم البحث في المصفوفة `arr`، التي تحتوي على أقصى عدد من عناصر الحجم، عن تكرار للرقم المخزن في القيمة. إذا تم العثور على الرقم، فسيتم إرجاع مؤشر المصفوفة الخاص به. وإلا، فسيتم إرجاع -1، مما يشير إلى عدم ظهور القيمة في المصفوفة.

```

int linearSearch (const int arr[], int size, int value)
{
    int index = 0;           // Used as a subscript to search the array
    int position = -1;       // To record the position of the search value
    bool found = false;      // Flag to indicate if the value was found

    while (index < size && !found)
    {
        if (arr[index] == value)    // If the value is found
        {
            found = true;           // Set the flag
            position = index;        // Record the value's subscript
        }
        index++;                   // Go to the next element
    }
    return position;              // Return the position, or -1
}

```



NOTE: The reason -1 is returned when the search value is not found in the array is because -1 is not a valid subscript.

البرنامج 8-1 هو برنامج متكامل يستخدم دالة البحث الخطي. يبحث البرنامج في اختبارات المصفوفات ذات العناصر الخمسة للحصول على درجة 100.

Program 8-1

```

1 // This program demonstrates the linear search algorithm.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype
6 int linearSearch(const int[], int, int);
7
8 int main()
9 {
10     const int SIZE = 5;
11     int tests[SIZE] = { 87, 75, 98, 100, 82 };
12     int results;
13
14     // Search the array for 100.
15     results = linearSearch(tests, SIZE, 100);
16
17     // If linearSearch returned -1, then 100 was not found.
18     if (results == -1)
19         cout << "You did not earn 100 points on any test\n";
20     else
21     {
22         // Otherwise results contains the subscript of
23         // the first 100 in the array.
24         cout << "You earned 100 points on test ";
25         cout << (results + 1) << endl;
26     }
27     return 0;
28 }
29

```

```

30 //*****
31 // The linearSearch function performs a linear search on an *
32 // integer array. The array arr, which has a maximum of size *
33 // elements, is searched for the number stored in value. If the *
34 // number is found, its array subscript is returned. Otherwise, *
35 // -1 is returned indicating the value was not in the array. *
36 //*****
37 int linearSearch(const int arr[], int size, int value)
38 {
39     int index = 0;          // Used as a subscript to search array
40     int position = -1;      // To record position of search value
41     bool found = false;    // Flag to indicate if the value was found
42
43     while (index < size && !found)
44     {
45         if (arr[index] == value)    // If the value is found
46         {
47             found = true;           // Set the flag
48             position = index;       // Record the value's subscript
49         }
50         index++;                  // Go to the next element
51     }
52     return position;             // Return the position, or -1
53 }

```

Program Output

You earned 100 points on test 4

The Binary Search

البحث الثنائي خوارزمية ذكية وأكثر كفاءة بكثير من البحث الخطي. متطلبها الوحيد هو ترتيب قيم المصفوفة. بدلاً من اختبار العنصر الأول في المصفوفة، تبدأ هذه الخوارزمية بالعنصر الأوسط. إذا احتوى هذا العنصر على القيمة المطلوبة، ينتهي البحث. وإلا، تكون القيمة في العنصر الأوسط إما أكبر من أو أصغر من القيمة المطلوب البحث عنها. إذا كانت أكبر، فسُتُعر على القيمة المطلوبة (إن كانت في القائمة) في مكان ما في النصف الأول من المصفوفة. أما إذا كانت أقل، فسُتُعر على القيمة المطلوبة (إن كانت في القائمة) في مكان ما في النصف الأخير من المصفوفة. في كلتا الحالتين، يُستبعد نصف عناصر المصفوفة من البحث الإضافي. إذا لم تُعر على القيمة المطلوبة في العنصر الأوسط، تُكرر العملية لنصف المصفوفة الذي يُحتمل أن يحتوي على القيمة. على سبيل المثال، إذا كان سيتم البحث في النصف الأخير من المصفوفة، تختبر الخوارزمية عنصرها الأوسط فوراً. إذا لم تُعر على القيمة المطلوبة، يُضيق البحث إلى ربع المصفوفة الذي يقع قبل أو بعد ذلك العنصر. تستمر هذه العملية حتى يتم العثور على القيمة المطلوبة، أو حتى لا توجد عناصر أخرى لاختبارها.

Here is the pseudocode for a function that performs a binary search on an array:

```
Set first to 0
Set last to the last subscript in the array
Set found to false
Set position to -1
While found is not true and first is less than or equal to last
    Set middle to the subscript halfway between array[first]
    and array[last]
    If array[middle] equals the desired value
        Set found to true
        Set position to middle
    Else If array[middle] is greater than the desired value
        Set last to middle - 1
    Else
        Set first to middle + 1
    End If
End While
Return position
```

تُستخدم الدالة `binarySearch` الموضحة في المثال التالي لإجراء بحث ثنائي على مصفوفة أعداد صحيحة. يتم البحث في المعامل الأول، `array`، الذي يحتوي على حد أقصى `numElems` من العناصر، عن تكرار للرقم المخزن في `value`. إذا تم العثور على الرقم، فسيتم إرجاع مؤشر المصفوفة الخاص به. وإلا، فسيتم إرجاع -1، مما يشير إلى عدم ظهور القيمة في المصفوفة.

```
int binarySearch(const int array[], int numElems, int value)
{
    int first = 0, // First array element
        last = numElems - 1, // Last array element
        middle, // Midpoint of search
        position = -1; // Position of search value
    bool found = false; // Flag

    while (!found && first <= last)
    {
        middle = (first + last) / 2; // Calculate midpoint
        if (array[middle] == value) // If value is found at mid
        {
            found = true;
            position = middle;
        }
        else if (array[middle] > value) // If value is in lower half
            last = middle - 1;
        else
            first = middle + 1; // If value is in upper half
    }
    return position;
}
```

البرنامج 8-2 هو برنامج متكامل يستخدم دالة البحث الثنائي. يبحث البرنامج في مصفوفة من أرقام تعريف الموظفين عن قيمة محددة.

Program 8-2

```
1 // This program demonstrates the binarySearch function, which
2 // performs a binary search on an integer array.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototype
7 int binarySearch(const int [], int, int);
8 const int SIZE = 20;
9
10 int main()
11 {
12     // Array with employee IDs sorted in ascending order.
13     int idNums[SIZE] = {101, 142, 147, 189, 199, 207, 222,
14                         234, 289, 296, 310, 319, 388, 394,
15                         417, 429, 447, 521, 536, 600};
16     int results; // To hold the search results
17     int empID;   // To hold an employee ID
18
19     // Get an employee ID to search for.
20     cout << "Enter the employee ID you wish to search for: ";
21     cin >> empID;
22
23     // Search for the ID.
24     results = binarySearch(idNums, SIZE, empID);
25
26     // If results contains -1 the ID was not found.
27     if (results == -1)
28         cout << "That number does not exist in the array. \n";
29     else
30     {
31         // Otherwise results contains the subscript of
32         // the specified employee ID in the array.
33         cout << "That ID is found at element " << results;
34         cout << " in the array.\n";
35     }
36     return 0;
37 }
38
39 /*******
40 // The binarySearch function performs a binary search on an *
41 // integer array. array, which has a maximum of size elements, *
42 // is searched for the number stored in value. If the number is *
43 // found, its array subscript is returned. Otherwise, -1 is *
44 // returned indicating the value was not in the array. *
45 /*******
46
```

```

47 int binarySearch(const int array[], int size, int value)
48 {
49     int first = 0,           // First array element
50     last = size - 1,        // Last array element
51     middle,                 // Midpoint of search
52     position = -1;          // Position of search value
53     bool found = false;     // Flag
54
55     while (!found && first <= last)
56     {
57         middle = (first + last) / 2;    // Calculate midpoint
58         if (array[middle] == value)    // If value is found at mid
59         {
60             found = true;
61             position = middle;
62         }
63         else if (array[middle] > value) // If value is in lower half
64             last = middle - 1;
65         else
66             first = middle + 1; // If value is in upper half
67     }
68     return position;
69 }

```

Program Output with Example Input Shown in Bold

Enter the employee ID you wish to search for: **199**
 That ID is found at element 4 in the array.



WARNING! Notice the array in Program 8-2 is initialized with its values already sorted in ascending order. The binary search algorithm will not work properly unless the values in the array are sorted.

7.11 Focus on Software Engineering: Introduction to Sorting Algorithms

المفهوم: يتم استخدام خوارزميات الفرز لترتيب البيانات في ترتيب معين.

سيقدم هذا القسم خوارزميتين بسيطتين للفرز: فرز الفقاعة والفرز بالتحديد.

The Bubble Sort

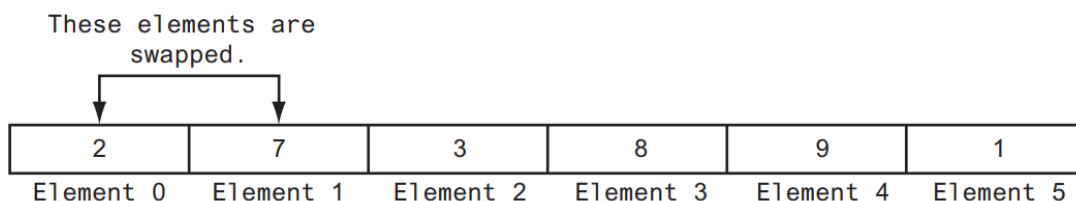
لنفترض أن لدينا المصفوفة الموضحة في الشكل 8-1. لنر كيف يُمكن استخدام فرز الفقاعات لترتيب عناصر المصفوفة تصاعديًا.

Figure 8-1 An array

7	2	3	8	9	1
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

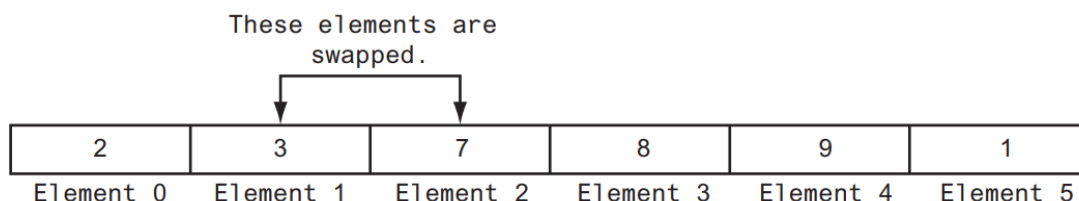
يبدأ فرز الفقاعات بمقارنة أول عنصرين في المصفوفة. إذا كان العنصر 0 أكبر من العنصر 1، فسيتم تبديلهما. ستظهر المصفوفة عندئذٍ كما هو موضح في الشكل 8-2.

Figure 8-2 Elements 0 and 1 are swapped



تُكرَّر هذه الطريقة مع العنصرين 1 و2. إذا كان العنصر 1 أكبر من العنصر 2، يُبدَّلان. ستظهر المصفوفة كما هو موضح في الشكل 8-3.

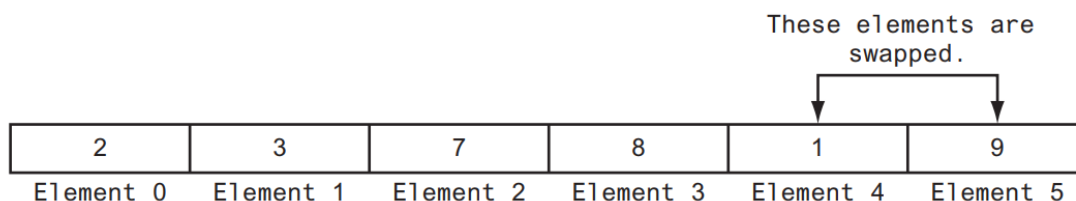
Figure 8-3 Elements 1 and 2 are swapped



بعد ذلك، تتم مقارنة العنصرين 2 و3. في هذه المصفوفة، تكون هذه العناصر بالترتيب الصحيح (العنصر 2 أصغر من العنصر 3)، لذا لا يتم تبديل أي قيم. مع استمرار الدورة، تتم مقارنة العنصرين 3 و4. مرة أخرى، ليس من الضروري تبديل القيم لأنها بالترتيب الصحيح.

ولكن عند مقارنة العنصرين 4 و5، يجب تبديلهما لأن العنصر 4 أكبر من العنصر 5. تظهر المصفوفة الآن كما هو موضح في الشكل 8-4.

Figure 8-4 Elements 4 and 5 are swapped

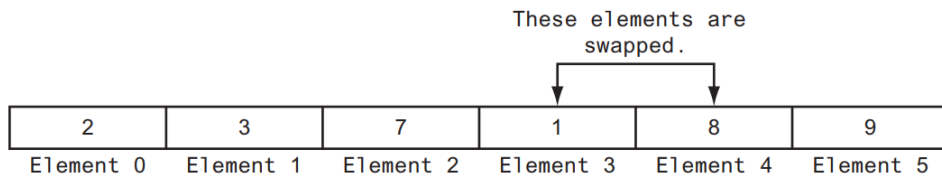


في هذه المرحلة، تكون المصفوفة بأكملها قد مُسحت مرة واحدة، وأكبر قيمة، 9، في الموضع الصحيح. مع ذلك، هناك عناصر أخرى لم تصل بعد إلى مواضعها النهائية.

لذا، ستقوم الخوارزمية بفحص المصفوفة مرة أخرى، مُقارنة كل عنصر بجاره. في الفحص التالي، ستتوقف المُقارنة بعد الوصول إلى العنصر قبل الأخير لأن العنصر الأخير يحتوي بالفعل على القيمة الصحيحة.

يبدأ الفحص الثاني بمقارنة العنصرين 0 و1. ولأن هذين العنصرين في الترتيب الصحيح، فلا يتم تبديلهما. تُقارن العناصر 1 و2 بعد ذلك، ولكن مرة أخرى، لا يتم تبديلهما. يستمر هذا حتى تتم مقارنة العنصرين 3 و4. ولأن العنصر 3 أكبر من العنصر 4، يتم تبديلهما. العنصر 4 هو آخر عنصر تتم مُقارنته خلال هذا الفحص، لذا يتوقف الفحص. تظهر المصفوفة الآن كما هو موضح في الشكل 8-5.

Figure 8-5 Elements 3 and 4 are swapped



في نهاية التمريرة الثانية، يحتوي العنصران الأخيران في المصفوفة على القيم الصحيحة.

تبدأ الآن التمريرة الثالثة، بمقارنة كل عنصر بجاره. مع ذلك، لن تشمل التمريرة الثالثة العنصرين الأخيرين، لأنهما قد تم فرزهما مسبقًا. عند انتهاء التمريرة الثالثة، ستحمل العناصر الثلاثة الأخيرة القيم الصحيحة، كما هو موضح في الشكل 8-6.

Figure 8-6 The array after the third pass

2	3	1	7	8	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

في كل مرة تمر فيها الخوارزمية عبر المصفوفة، يُقلص حجم الجزء الممسوح بمقدار عنصر واحد، ونُقل أكبر قيمة في الجزء الممسوح إلى موضعها النهائي. بعد إتمام جميع عمليات المرور، ستظهر المصفوفة كما هو موضح في الشكل 8-7.

Figure 8-7 The array with all elements sorted

1	2	3	7	8	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

Here is the bubble sort algorithm in pseudocode:

```
For maxElement = each subscript in the array, from the last to the first
  For index = 0 To maxElement - 1
    If array[index] > array[index + 1]
      swap array[index] with array[index + 1]
    End If
  End For
End For
```

وهذا هو كود C++ لخوارزمية فرز الفقاعات، مكتوبًا كدالة تُفرز مصفوفة من الأعداد الصحيحة. تقبل الدالة وسيطتين: المصفوفة المراد فرزها، وحجم المصفوفة.

```
1 void bubbleSort(int array[], int size)
2 {
3     int maxElement;
4     int index;
5
6     for (maxElement = size - 1; maxElement > 0; maxElement--)
7     {
8         for (index = 0; index < maxElement; index++)
9         {
10             if (array[index] > array[index + 1])
11             {
12                 swap(array[index], array[index + 1]);
13             }
14         }
15     }
16 }
```

يوضح البرنامج 4-8 التابع bubbleSort في برنامج كامل.

Program 8-4

```
1 // This program demonstrates the Bubble Sort algorithm.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototypes
6 void bubbleSort(int[], int);
7 void swap(int &, int &);
8
9 int main()
10 {
11     const int SIZE = 6;
12
13     // Array of unsorted values
14     int values[SIZE] = { 6, 1, 5, 2, 4, 3 };
15
16     // Display the unsorted array.
17     cout << "The unsorted values:\n";
18     for (auto element : values)
19         cout << element << " ";
20     cout << endl;
21
22     // Sort the array.
23     bubbleSort(values, SIZE);
24
25     // Display the sorted array.
26     cout << "The sorted values:\n";
27     for (auto element : values)
28         cout << element << " ";
29     cout << endl;
30
31     return 0;
32 }
33
34 //*****
35 // The bubbleSort function sorts an int array in ascending order. *
36 //*****
37 void bubbleSort(int array[], int size)
38 {
39     int maxElement;
40     int index;
41
42     for (maxElement = size - 1; maxElement > 0; maxElement--)
43     {
44         for (index = 0; index < maxElement; index++)
45         {
46             if (array[index] > array[index + 1])
47             {
48                 swap(array[index], array[index + 1]);
49             }
50         }
51     }
52 }
53
```

```

54 //*****
55 // The swap function swaps a and b in memory.      *
56 //*****
57 void swap(int &a, int &b)
58 {
59     int temp = a;
60     a = b;
61     b = temp;
62 }

```

Program Output

The unsorted values:

6 1 5 2 4 3

The sorted values:

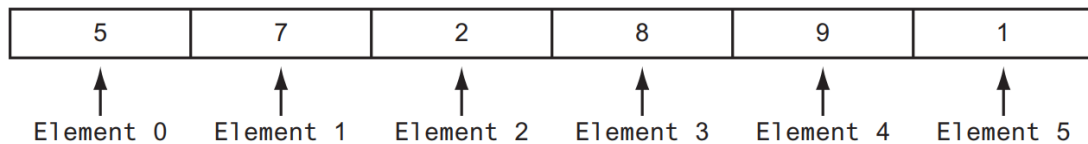
1 2 3 4 5 6

The Selection Sort Algorithm

يعمل فرز التحديد على النحو التالي: يتم تحديد أصغر قيمة في المصفوفة ونقلها إلى العنصر 0. ثم يتم تحديد أصغر قيمة تالية ونقلها إلى العنصر 1. تستمر هذه العملية حتى يتم وضع جميع العناصر في ترتيبها الصحيح.

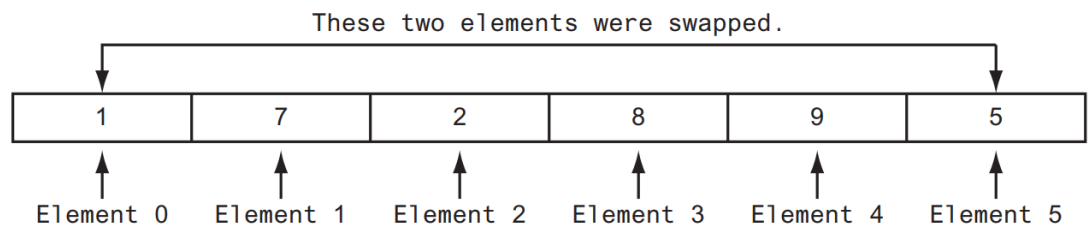
دعونا نرى كيف يعمل فرز التحديد عند ترتيب عناصر المصفوفة في الشكل 8-8.

Figure 8-8 Values in an array



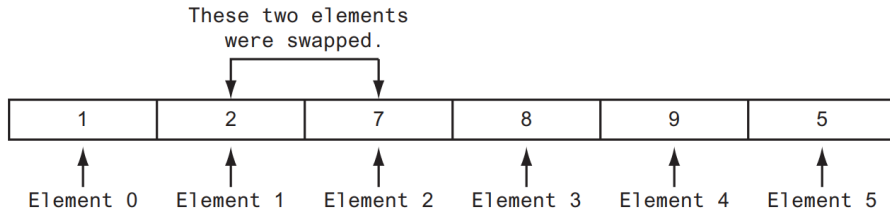
يقوم فرز التحديد بمسح المصفوفة، بدءًا من العنصر 0، وتحديد العنصر ذي القيمة الأصغر. بعد ذلك، تُبدّل محتويات هذا العنصر بمحتويات العنصر 0. في هذا المثال، يُبدّل الرقم 1 المُخزّن في العنصر 5 بالرقم 5 المُخزّن في العنصر 0. بعد الاستبدال، تظهر المصفوفة كما هو موضح في الشكل 8-9.

Figure 8-9 Values in the array after the first swap



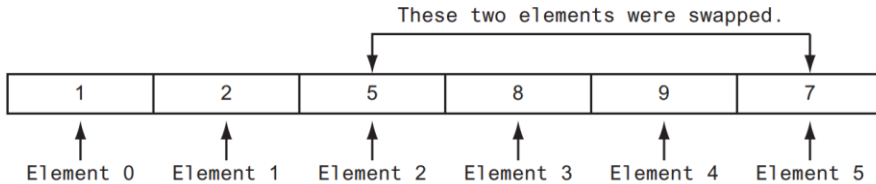
بعد ذلك، تُكرر الخوارزمية العملية، ولكن بما أن العنصر 0 يحتوي بالفعل على أصغر قيمة في المصفوفة، يُمكن استبعاده من الإجراء. هذه المرة، تبدأ الخوارزمية المسح من العنصر 1. في هذا المثال، تُبدل القيمة في العنصر 2 بالقيمة في العنصر 1. بعد ذلك، تظهر المصفوفة كما هو موضح في الشكل 8-10.

Figure 8-10 Values in the array after the second swap



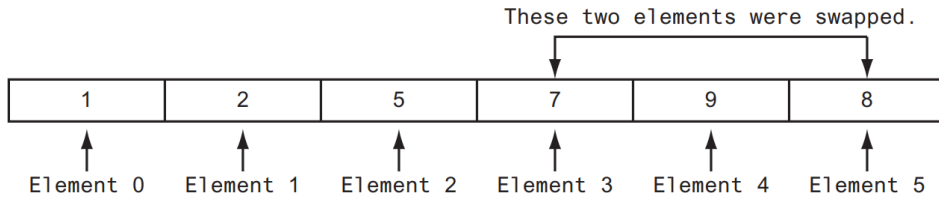
تُكرر العملية مرة أخرى، ولكن هذه المرة يبدأ المسح من العنصر 2. ستجد الخوارزمية أن العنصر 5 يحتوي على أصغر قيمة تالية. تُستبدل قيمة هذا العنصر بقيمة العنصر 2، مما يؤدي إلى ظهور المصفوفة كما هو موضح في الشكل 8-11.

Figure 8-11 Values in the array after the third swap



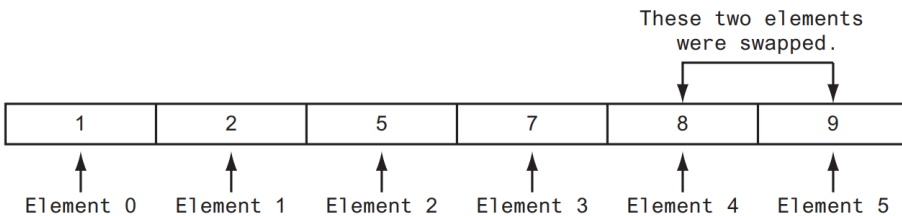
بعد ذلك، يبدأ المسح عند العنصر 3. يتم تبديل قيمته بقيمة العنصر 5، مما يؤدي إلى ظهور المصفوفة كما هو موضح في الشكل 8-12.

Figure 8-12 Values in the array after the fourth swap



في هذه المرحلة، يتبقى عنصران فقط للفرز. وجدت الخوارزمية أن قيمة العنصر 5 أصغر من قيمة العنصر 4، لذا تم تبديلهما. هذا يضع المصفوفة في ترتيبها النهائي، كما هو موضح في الشكل 8-13.

Figure 8-13 Values in the array after the fifth swap



يوضح البرنامج 5-8 وظيفة selectionSort في برنامج كامل.

Program 8-5

```
1 // This program demonstrates the Selection Sort algorithm.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototypes
6 void selectionSort(int[], int);
7 void swap(int &, int &);
8
9 int main()
10 {
11     const int SIZE = 6;
12
13     // Array of unsorted values
14     int values[SIZE] = { 6, 1, 5, 2, 4, 3 };
15
16     // Display the unsorted array.
17     cout << "The unsorted values:\n";
18     for (auto element : values)
19         cout << element << " ";
20     cout << endl;
21
22     // Sort the array.
23     selectionSort(values, SIZE);
24
25     // Display the sorted array.
26     cout << "The sorted values:\n";
27     for (auto element : values)
28         cout << element << " ";
29     cout << endl;
30
31     return 0;
32 }
33
34 //*****
35 // The selectionSort function sorts an int array in ascending order. *
36 //*****
37 void selectionSort(int array[], int size)
38 {
39     int minIndex, minValue;
40
41     for (int start = 0; start < (size - 1); start++)
42     {
43         minIndex = start;
44         minValue = array[start];
45         for (int index = start + 1; index < size; index++)
46         {
47             if (array[index] < minValue)
48             {
49                 minValue = array[index];
50                 minIndex = index;
51             }
52         }
53         swap(array[minIndex], array[start]);
54     }
55 }
56
57 //*****
58 // The swap function swaps a and b in memory. *
59 //*****
60 void swap(int &a, int &b)
61 {
```

Program 8-5*(continued)*

```

62     int temp = a;
63     a = b;
64     b = temp;
65 }

```

Program Output

```

The unsorted values:
6 1 5 2 4 3
The sorted values:
1 2 3 4 5 6

```

7.12 Sorting and Searching vectors

المفهوم: يمكن تطبيق خوارزميات الفرز والبحث التي درستها في هذا الفصل على متجهات STL وكذلك المصفوفات.

يوضح البرنامج 7-8 كيفية استخدام خوارزميات الفرز بالاختيار والبحث الثنائي مع متجه من السلاسل.

Program 8-7

```

1  // This program demonstrates how to sort and search a
2  // string vector using selection sort and binary search.
3  #include <iostream>
4  #include <string>
5  #include <vector>
6  using namespace std;
7
8  // Function prototypes
9  void selectionSort(vector<string>&);
10 void swap(string &, string &);
11 int binarySearch(const vector<string>&, string);
12
13 int main()
14 {
15     string searchValue; // Value to search for
16     int position;       // Position of found value
17
18     // Define a vector of strings.
19     vector<string> names{ "Lopez", "Smith", "Pike", "Jones",
20                          "Abernathy", "Hall", "Wilson", "Kimura",
21                          "Alvarado", "Harrison", "Geddes", "Irvine" };
22
23     // Sort the vector.
24     selectionSort(names);
25
26     // Display the vector's elements.
27     cout << "Here are the sorted names:\n";
28     for (auto element : names)
29         cout << element << endl;
30     cout << endl;
31
32     // Search for a name.
33     cout << "Enter a name to search for: ";
34     getline(cin, searchValue);
35     position = binarySearch(names, searchValue);

```

Program 8-7*(continued)*

```
36
37     // Display the results.
38     if (position != -1)
39         cout << "That name is found at position " << position << endl;
40     else
41         cout << "That name is not found.\n";
42
43     return 0;
44 }
45
46 //*****
47 // The selectionSort function sorts a string vector in ascending order.*
48 //*****
49 void selectionSort(vector<string> &v)
50 {
51     int minIndex;
52     string minValue;
53
54     for (int start = 0; start < (v.size() - 1); start++)
55     {
56         minIndex = start;
57         minValue = v[start];
58         for (int index = start + 1; index < v.size(); index++)
59         {
60             if (v[index] < minValue)
61             {
62                 minValue = v[index];
63                 minIndex = index;
64             }
65         }
66         swap(v[minIndex], v[start]);
67     }
68 }
69
70 //*****
71 // The swap function swaps a and b in memory.*
72 //*****
73 void swap(string &a, string &b)
74 {
75     string temp = a;
76     a = b;
77     b = temp;
78 }
79
80 //*****
81 // The binarySearch function performs a binary search on a      *
82 // string vector. array, which has a maximum of size elements, *
83 // is searched for the number stored in value. If the number is *
84 // found, its vector subscript is returned. Otherwise, -1 is     *
85 // returned indicating the value was not in the vector.         *
86 //*****
```

```

87
88 int binarySearch(const vector<string> &v, string str)
89 {
90     int first = 0,           // First vector element
91         last = v.size() - 1, // Last vector element
92         middle,              // Mid point of search
93         position = -1;       // Position of search value
94     bool found = false;      // Flag
95
96     while (!found && first <= last)
97     {
98         middle = (first + last) / 2; // Calculate mid point
99         if (v[middle] == str)        // If value is found at mid
100         {
101             found = true;
102             position = middle;
103         }
104         else if (v[middle] > str)     // If value is in lower half
105             last = middle - 1;
106         else
107             first = middle + 1;      // If value is in upper half
108     }
109     return position;
110 }

```

Program Output with Example Input Shown in Bold

Here are the sorted names:

Abernathy
 Alvarado
 Geddes
 Hall
 Harrison
 Irvine
 Jones
 Kimura
 Lopez
 Pike
 Smith
 Wilson

Enter a name to search for: **Lopez**

That name is found at position 8

Program Output with Example Input Shown in Bold

Here are the sorted names:

Abernathy
 Alvarado
 Geddes
 Hall

Program 8-7*(continued)*

Harrison
Irvine
Jones
Kimura
Lopez
Pike
Smith
Wilson

Enter a name to search for: **Carter**

That name is not found.

انتهت المحاضرة السابعة